

Insights into DeepSeek-V3: Scaling Challenges and Reflections on Hardware for AI Architectures

Chenggang Zhao DeepSeek-AI Beijing, China chenggangz@deepseek.com	Chengqi Deng DeepSeek-AI Beijing, China cq.deng@deepseek.com	Chong Ruan DeepSeek-AI Beijing, China chong.ruan@deepseek.com
Damai Dai DeepSeek-AI Beijing, China damai.dai@deepseek.com	Huazuo Gao DeepSeek-AI Beijing, China gaohuazuo@deepseek.com	Jiashi Li DeepSeek-AI Beijing, China js.li@deepseek.com
Liyue Zhang* DeepSeek-AI Beijing, China ly.zhang@deepseek.com	Panpan Huang DeepSeek-AI Beijing, China pp.huang@deepseek.com	Shangyan Zhou DeepSeek-AI Beijing, China sy.zhou@deepseek.com
Shirong Ma DeepSeek-AI Beijing, China mashirong.2000@deepseek.com	Wenfeng Liang DeepSeek-AI Beijing, China wenfeng.liang@deepseek.com	Ying He DeepSeek-AI Beijing, China ying.he@deepseek.com
Yuqing Wang* DeepSeek-AI Beijing, China wangyq@deepseek.com	Yuxuan Liu DeepSeek-AI Beijing, China liuyuxuan@deepseek.com	Y.X. Wei DeepSeek-AI Beijing, China weiyx@deepseek.com

Abstract

The rapid scaling of large language models (LLMs) has unveiled critical limitations in current hardware architectures, including constraints in memory capacity, computational efficiency, and interconnection bandwidth. DeepSeek-V3, trained on 2,048 NVIDIA H800 GPUs, demonstrates how hardware-aware model co-design can effectively address these challenges, enabling cost-efficient training and inference at scale. This paper presents an in-depth analysis of the DeepSeek-V3/R1 model architecture and its AI infrastructure, highlighting key innovations such as Multi-head Latent Attention (MLA) for enhanced memory efficiency, Mixture of Experts (MoE) architectures for optimized computation-communication trade-offs, FP8 mixed-precision training to unlock the full potential of hardware capabilities, and a Multi-Plane Network Topology to minimize

cluster-level network overhead. Building on the hardware bottlenecks encountered during DeepSeek-V3’s development, we engage in a broader discussion with academic and industry peers on potential future hardware directions, including precise low-precision computation units, scale-up and scale-out convergence, and innovations in low-latency communication fabrics. These insights underscore the critical role of hardware and model co-design in meeting the escalating demands of AI workloads, offering a practical blueprint for innovation in next-generation AI systems.

CCS Concepts

• Computer systems organization → Architectures.

Keywords

Large Language Model, Mixture-of-Experts, Deep Learning, FP8 Mixed-Precision Training, Multi-Plane Network, Co-Design

ACM Reference Format:

Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Huazuo Gao, Jiashi Li, Liyue Zhang, Panpan Huang, Shangyan Zhou, Shirong Ma, Wenfeng Liang, Ying He, Yuqing Wang, Yuxuan Liu, Y.X. Wei . 2025. Insights into DeepSeek-V3: Scaling Challenges and Reflections on Hardware for AI Architectures. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA ’25)*, June 21–25, 2025, Tokyo, Japan. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3695053.3731412>

洞察 DeepSeek-V3：扩展挑战与对人工智能架构硬件的反思

赵成钢 深度求索 北京, 中国 chenggangz@deepseek.com	邓承祺 深度求索 北京, 中国 cq.deng@deepseek.com	阮冲 深度求索 北京, 中国 chong.ruan@deepseek.com
戴大麦 深度求索 北京, 中国 damai.dai@deepseek.com	高华祚 深度求索 北京, 中国 gaohuazuo@deepseek.com	李佳实 深度求索 北京, 中国 js.li@deepseek.com
张立越* 深度求索 北京, 中国 ly.zhang@deepseek.com	黄盼盼 深度求索 北京, 中国 pp.huang@deepseek.com	周上炎 深度求索 北京, 中国 sy.zhou@deepseek.com
马世荣 深度求索 北京, 中国 mashirong.2000@deepseek.com	梁文锋 深度求索 北京, 中国 wenfeng.liang@deepseek.com	何颖 深度求索 北京, 中国 ying.he@deepseek.com
王雨晴* 深度求索 北京, 中国 wangyq@deepseek.com	刘宇轩 深度求索 北京, 中国 liuyuxuan@deepseek.com	Y.X. Wei 深度求索 北京, 中国 weiyx@deepseek.com

Abstract

大型语言模型（LLMs）的快速扩展揭示了当前硬件架构的关键局限性,包括内存容量、计算效率和互连带宽

ISCA ’25, 2025 年 6 月 21–25 日, 日本东京

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1261-6/2025/06

<https://doi.org/10.1145/3695053.3731412>

This is the author’s version of the work. It is posted here for your personal use. Not for redistribution.
The definitive version appeared as part of the Industry Track in *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA ’25)*.

1 Introduction

1.1 Background

Large Language Models (LLMs) have undergone rapid evolution in recent years, driven by iterative advancements in model design, computational power, and data availability. In 2024, groundbreaking models such as GPT4o [59], LLaMa-3 [3], Claude 3.5 Sonnet [8], Grok-2 [74], Qwen2.5 [71], Gemini-2 [37] and our DeepSeek-V3 [26] have showcased remarkable progress, further narrowing the gap towards Artificial General Intelligence (AGI). As the Scaling Laws [45] shows, increasing model size, training data, and computational resources leads to substantial improvements in model performance, underscoring the pivotal role of scaling in advancing AI capabilities. Collectively, these developments have ushered in an era where scaling model size and computational power is seen as the key to unlocking higher levels of intelligence.

Recent developments, reasoning models such as OpenAI's o1/o3 series models [60, 61], DeepSeek-R1 [28], Claude-3.7 Sonnet [9], Gemini 2.5 Pro [38], Seed1.5-Thinking [68] and Qwen3 [72] have demonstrated not only the benefits conferred by large-scale architectures, but also the necessity of improving inference efficiency, particularly in handling longer contexts and achieving greater reasoning depth. These advancements underscore the need for faster and more efficient inference, consequently placing ever-increasing demands on computational resources.

To meet these challenges, industry leaders such as Alibaba, ByteDance, Google, xAI and Meta have deployed colossal training clusters [33, 42, 43, 56, 62, 75], featuring tens or even hundreds of thousands of GPUs or TPUs. While such massive infrastructures have enabled the development of state-of-the-art models, their exorbitant costs present significant barriers for smaller research teams and organizations. Despite these barriers, open-source startups such as DeepSeek [23–26, 28] and Mistral [41, 55] are also striving to develop state-of-the-art models. Among them, DeepSeek has especially demonstrated that effective software-hardware co-design can enable cost-efficient training of large models, leveling the playing field for smaller teams.

Building on this tradition, DeepSeek-V3 [26] represents a new milestone in cost-effective training. By leveraging just 2,048 NVIDIA H800 GPUs, DeepSeek-V3 achieves state-of-the-art performance. This achievement aligns with the commitment to advance AI through practical and scalable solutions, as previously demonstrated in the cost-effective architecture of Fire-Flyer AI-HPC [7]. The practices and insights derived from DeepSeek-V3 demonstrate how existing hardware resources can be harnessed to their fullest potential, offering valuable lessons for the broader AI and HPC communities.

1.2 Objectives

This paper does not aim to reiterate the detailed architectural and algorithmic specifics of DeepSeek-V3, which are extensively documented in its technical report [26]. Instead, it adopts a dual perspective—spanning hardware architecture and model design—to explore the intricate interplay between them in achieving cost-efficient large-scale training and inference. By examining this synergy, we aim to provide actionable insights for scaling LLMs efficiently without sacrificing performance or accessibility.

Specifically, the paper focuses on:

- **Hardware-Driven Model Design:** Analyze how hardware features, such as FP8 low-precision computation and scale-up/scale-out network properties, informed the architectural choices in DeepSeek-V3.
- **Mutual Dependencies Between Hardware and Models:** Investigate how hardware capabilities shape model innovation and how the evolving demands of LLMs drive the need for next-generation hardware.
- **Future Directions for Hardware Development:** Derive actionable insights from DeepSeek-V3 to guide the co-design of future hardware and model architectures, paving the way for scalable, cost-efficient AI systems.

1.3 Structure of this Paper

The remainder of this paper is organized as follows. Section 2 explores the design principles underpinning DeepSeek-V3 model architecture, highlighting key innovations such as Multi-head Latent Attention, Mixture-of-Experts optimizations and Multi-Token Prediction Module. Section 3 illustrates how our model architecture pursues low-precision computation and communication. Section 4 includes scale-up interconnection optimizations, discusses scale-up/scale-out convergence, and explores how hardware features influence parallelism and expert selection strategies. Section 5 focuses on scale-out network optimizations, including multi-plane network co-designs and low-latency interconnects. Besides current limitations and future suggestions mentioned in Section 3~5, Section 6 elaborates on more critical insights from DeepSeek-V3, and identifies directions for future hardware and model co-design.

2 Design Principles for DeepSeek Models

The development of DeepSeek-V3 exemplifies a hardware-aware approach to scaling LLMs, where each design decision was carefully aligned with hardware constraints to optimize performance and cost efficiency.

As shown in Figure 1, DeepSeek-V3 employs the **DeepSeek-MoE** [27] and **Multi-head Latent Attention (MLA)** [25] architectures that have been proven effective in DeepSeek-V2 [25]. DeepSeek-MoE unlocks the potential of MoE architecture, while MLA drastically reduces memory consumption by compressing Key-Value (KV) caches. In addition, **DeepSeek-V3** incorporates **FP8 mixed-precision training**, significantly lowering computational costs and making large-scale training more practical without compromising model quality. To improve the inference speed, DeepSeek-V3 integrates speculative decoding based on its **Multi-Token Prediction Module**, which significantly increases the generation speed. Beyond model architecture, we also explored cost-efficient AI infrastructure by deploying a **Multi-Plane** two-layer Fat-Tree network to replace a traditional three-layer Fat-Tree topology, reducing cluster networking costs.

These innovations aim to address three core challenges in scaling LLMs—**memory efficiency**, **cost-effectiveness**, and **inference speed**—which are explored in detail in the following subsections.

2.1 Memory Efficiency

LLMs generally require significant memory resources, with memory demands increasing by more than 1000% per year. In contrast, the

方面的约束。在 2,048 块 NVIDIA H800 GPU 上训练的 DeepSeek-V3 展示了硬件感知的模型协同设计如何有效应对这些挑战, 实现大规模、高性价比的训练和推理。本文深入分析了 DeepSeek-V3/R1 模型架构及其人工智能基础设施, 重点介绍了多项关键创新, 例如用于提升内存效率的多头潜在注意力 (MLA)、用于优化计算-通信权衡的专家混合 (MoE) 架构、用于充分释放硬件潜力的 FP8 混合精度训练, 以及用于最小化集群级网络开销的多平面网络拓扑。基于 DeepSeek-V3 开发过程中遇到的硬件瓶颈, 我们与学术界和工业界的同行进行了更广泛的讨论, 探讨了未来硬件的潜在发展方向, 包括精确的低精度计算单元、纵向扩展与横向扩展的融合, 以及低延迟通信架构的创新。这些见解强调了硬件与模型协同设计在满足人工智能工作负载日益增长需求方面的关键作用, 为下一代人工智能系统的创新提供了实用的蓝图。

CCS Concepts

- 计算机系统组织 → 体系结构.

Keywords

大语言模型, 专家混合, 深度学习, FP8 混合精度训练, 多平面网络, 协同设计

ACM Reference Format:

赵成钢, 邓承祺, 阮冲, 戴大麦, 高华祚, 李佳实, 张立越, 黄盼盼, 周尚炎, 马世荣, 梁文峰, 何颖, 王雨晴, 刘宇轩, Y.X. Wei . 2025. 洞察 DeepSeek-V3: 扩展挑战与对人工智能架构硬件的反思. In 第 52 届计算机体系结构国际研讨会论文集 (ISCA '25), 2025 年 6 月 21–25 日, 日本东京. ACM, New York, NY, USA, 22 pages. <https://doi.org/10.1145/3695053.3731412>

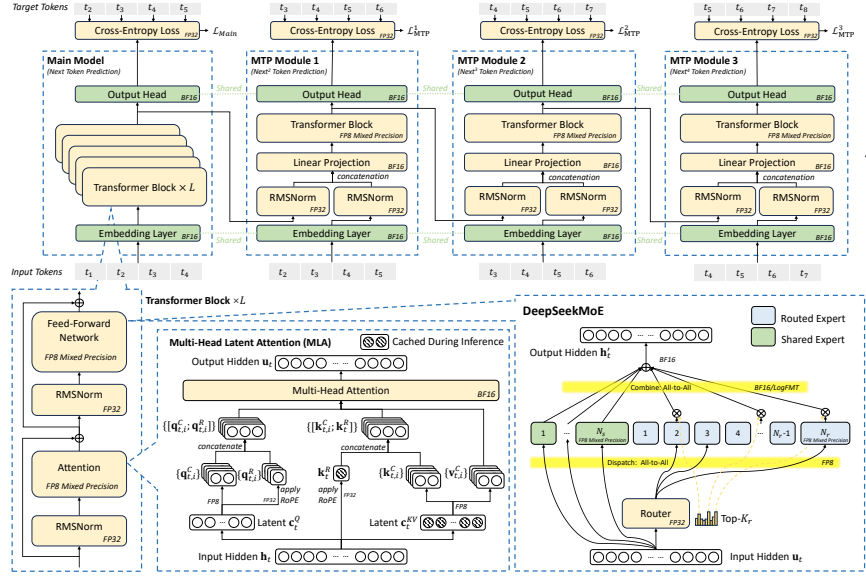


Figure 1: Basic architecture of DeepSeek-V3. Built upon DeepSeek-V2’s MLA and DeepSeekMoE, a Multi-Token Prediction Module and FP8 mixed-precision training are introduced to enhance inference and training efficiency. The figure indicates the precision used for computations in different parts of the architecture. All components take inputs and outputs in BF16.

growth rate of high-speed memory (e.g., HBM) capacity is much slower, typically less than 50% per year [35]. While multi-node parallelism is a viable solution to address memory limitations, optimizing memory usage at the source remains a crucial and effective strategy.

2.1.1 Low-Precision Models. Compared to models that utilize BF16 for weights, FP8 significantly reduces memory consumption by half, effectively alleviating the AI memory wall challenge. A detailed discussion of low-precision techniques is provided in Section 3 Low-Precision Driven Design.

2.1.2 Reducing KV Cache with MLA. For LLM inference, user requests often involve multi-turn conversations. To handle these efficiently, the context from previous requests is cached in what is commonly referred to as the *KV cache*. KV cache addresses this challenge by caching the **Key** and **Value** vectors of previously processed tokens, eliminating the need to recompute them for subsequent tokens. During each inference step, the model only computes the Key and Value vectors for the current token and performs attention computation by combining them with the cached Key-Value pairs from the history. This incremental computation reduces the complexity of generating each token to $O(N)$, making it efficient when processing long sequences or multi-turn inputs. However, it introduces

a memory-bound bottleneck because the computation shifts from GEMM to GEMV, which has a much lower compute-to-memory ratio. With modern hardware offering hundreds of TFLOPS, GEMV quickly becomes limited by memory bandwidth, making memory access the primary bottleneck.

To address this bottleneck, we employ **Multi-head Latent Attention (MLA)** [25] that compresses the KV representations of all attention heads into a smaller latent vector using a projection matrix, which is jointly trained with the model. During inference, only the latent vector needs to be cached, significantly reducing memory consumption compared to storing the KV cache for all attention heads.

In addition to MLA, several other approaches have been proposed to reduce the size of the KV cache. These methods are highly valuable and provide significant inspiration for advancements in memory-efficient attention mechanisms:

- **Shared KV (Grouped-Query Attention, GQA; Multi-Query Attention, MQA):** Instead of maintaining separate KV pairs for each attention head, multiple heads share a single set of KV pairs, significantly compressing KV storage. Representative methods include GQA [5] and MQA [70].

1 Introduction

1.1 Background

近年来, 大型语言模型 (LLMs) 在模型设计、计算能力和数据可用性方面迭代进步的推动下, 经历了快速的演变。2024 年, 诸如 GPT-4o [59]、LLaMa-3 [3]、Claude 3.5 Sonnet [8]、Grok-2 [74]、Qwen2.5 [71]、Gemini-2 [37] 以及我们的 DeepSeek-V3 [26] 等突破性模型展现了显著的进展, 进一步缩小了与通用人工智能 (AGI) 的差距。正如扩展定律 [45] 所示, 增加模型规模、训练数据和计算资源会带来模型性能的实质性提升, 突显了扩展在推进人工智能能力方面的关键作用。总的来说, 这些发展开启了一个时代, 其中扩展模型规模和计算能力被视为解锁更高智能水平的关键。

近期的发展中, 推理模型如 OpenAI 的 o1/o3 系列模型 [60, 61]、DeepSeek-R1 [28]、Claude-3.7 Sonnet [9]、Gemini 2.5 Pro [38]、Seed1.5-Thinking [68] 和 Qwen3 [72] 不仅展示了大规模架构带来的益处, 也凸显了提高推理效率的必要性, 特别是在处理更长上下文和实现更深层次推理方面。这些进步强调了对更快、更高效推理的需求, 从而对计算资源提出了日益增长的要求。

为了应对这些挑战, 行业领导者如阿里巴巴、字节跳动、谷歌、xAI 和 Meta 已经部署了庞大的训练集群 [33, 42, 43, 56, 62, 75], 配备了数万甚至数十万的 GPU 或 TPU。虽然如此庞大的基础设施使得开发最先进的模型成为可能, 但其高昂的成本为较小的研究团队和组织设置了重大障碍。尽管存在这些障碍, 像 DeepSeek [23–26, 28] 和 Mistral [41, 55] 这样的开源初创公司也在努力开发最先进的模型。其中, DeepSeek 尤其证明了有效的软硬件协同设计可以实现大型模型的经济高效训练, 为较小的团队创造了公平的竞争环境。

秉承这一传统, DeepSeek-V3 [26] 代表了经济高效训练的新里程碑。仅利用 2,048 个 NVIDIA H800 GPU, DeepSeek-V3 就实现了最先进的性能。这一成就符合通过实用且可扩展的解决方案推进人工智能的承诺, 正如先前在 Fire-Flyer AI-HPC [7] 的经济高效架构中所展示的那样。从 DeepSeek-V3 中获得的实践经验和见

解展示了如何最大限度地利用现有硬件资源, 为更广泛的人工智能和高性能计算社区提供了宝贵的经验。

1.2 Objectives

本文无意重述 DeepSeek-V3 详细的架构与算法细节, 这些内容已在技术报告 [26] 中详尽记录。相反, 本文采用硬件架构与模型设计的双重视角, 探讨两者之间为实现高性价比的大规模训练与推理而存在的复杂相互作用。通过审视这种协同作用, 我们旨在为高效扩展大语言模型提供可行的见解, 同时不牺牲性能或可及性。

具体而言, 本文聚焦于:

- **Hardware-Driven Model Design:** 分析硬件特性 (如 FP8 低精度计算及纵向扩展/横向扩展网络特性) 如何影响了 DeepSeek-V3 的架构选择。
- **Mutual Dependencies Between Hardware and Models:** 探究硬件能力如何塑造模型创新, 以及大语言模型不断演进的需求如何驱动对下一代硬件的需求。
- **Future Directions for Hardware Development:** 从 DeepSeek-V3 中推导出可行的见解, 以指导未来硬件与模型架构的协同设计, 为可扩展、高性价比的人工智能系统铺平道路。

1.3 Structure of this Paper

本文其余部分组织如下。第 2 节探讨支撑 DeepSeek-V3 模型架构的设计原则, 重点介绍关键创新, 如多头潜在注意力、混合专家优化和多令牌预测模块。第 3 节阐述我们的模型架构如何追求低精度计算与通信。第 4 节包含纵向扩展互连优化, 讨论纵向扩展/横向扩展的融合, 并探讨硬件特性如何影响并行策略与专家选择策略。第 5 节聚焦于横向扩展网络优化, 包括多平面网络协同设计和低延迟互连。除了第 3–5 节提到的当前局限性与未来建议外, 第 7 节详细阐述了从 DeepSeek-V3 中获得的更多关键见解, 并指出了未来硬件与模型协同设计的方向。

Table 1: KV cache size comparison (BF16 precision): DeepSeek-V3 (MLA) largely reduces KV cache size compared to other models using GQA.

Model	KV Cache Per Token	Multiplier
DeepSeek-V3 (MLA)	70.272 KB	1x
Qwen-2.5 72B (GQA)	327.680 KB	4.66x
LLaMa-3.1 405B (GQA)	516.096 KB	7.28x

- **Windowed KV:** For long sequences, only a sliding window of KV pairs is retained in the cache, discarding results outside the window. While this reduces storage, it compromises long-context reasoning. Representative methods include Long-former [11] and related architectures.
- **Quantized Compression:** KV pairs are stored using low-bit representations [40, 44, 52], further reducing memory usage. Quantization achieves significant compression with minimal impact on model performance.

Table 1 compares the KV cache memory usage per token among DeepSeek-V3, Qwen-2.5 72B [71], and LLaMa-3.1 405B [4]. By adopting MLA, DeepSeek-V3 achieves a significant reduction in KV cache size, requiring only 70 KB per token, substantially less than LLaMa-3.1 405B’s 516 KB and Qwen-2.5 72B’s 327 KB. This reduction highlights the efficiency of MLA in compressing KV representations compared to GQA-based methods. The ability to achieve such a significant reduction in memory consumption makes DeepSeek-V3 particularly well-suited for scenarios involving long-context processing and resource-constrained environments, enabling more scalable and cost-effective inference.

2.1.3 Future Directions and Perspectives on Resource-Efficient Techniques. While reducing the size of the KV cache is a promising method for improving memory efficiency, the quadratic complexity inherent in Transformer-based autoregressive decoding remains a formidable challenge, especially for extremely long contexts. Recent research efforts, such as Mamba-2 [21] and Lightning Attention [63], investigate linear-time alternatives that offer new possibilities for balancing computational cost and model performance. In addition, approaches such as sparse attention [76], which seek to compress and sparsely activate attention keys and values, represent another attempt at overcoming the computational challenges associated with attention. We look forward to collaborative progress with the broader community toward breakthroughs in this area.

2.2 Cost-Effectiveness of MoE Models

For sparse computing, we have developed DeepSeekMoE, an advanced **Mixture of Experts (MoE)** architecture, which is illustrated in the lower right part of Figure 1. The advantages of MoE models lie in two folds.

2.2.1 Reducing Computational Requirements for Training. The primary advantage of the MoE architecture lies in its ability to significantly reduce training costs. By selectively activating only a subset of expert parameters, MoE models allow the total parameter count to scale up dramatically while keeping computational requirements modest. For example, **DeepSeek-V2** features 236B parameters, but only 21B parameters are activated per token. Similarly, **DeepSeek-V3** expands to 671B parameters—nearly three

Table 2: Comparison of computational costs for training MoE and dense models: Computational cost per token is measured, assuming a sequence length of 4096.

Model	Size	Training Cost
DeepSeek-V2 MoE	236B	155 GFLOPS/Token
DeepSeek-V3 MoE	671B	250 GFLOPS/Token
Qwen-72B Dense	72B	394 GFLOPS/Token
LLaMa-405B Dense	405B	2448 GFLOPS/Token

times the size of V2—while keeping the activation per token at just 37B. In comparison, dense models such as Qwen2.5-72B and LLaMa3.1-405B require all parameters to be active during training.

As shown in Table 2, the total computational cost for DeepSeek-V3 is approximately 250 GFLOPS per token, whereas the 72B dense model requires 394 GFLOPS and the 405B dense model requires 2448 GFLOPS. This demonstrates that MoE models achieve comparable or even superior performance to dense models while consuming an order of magnitude less computational resources.

2.2.2 Advantages for Personal Use and On-Premises Deployment. In a future where personalized LLM agents [53] become ubiquitous, MoE models offer unique advantages in single-request scenarios. Because only a subset of parameters is activated per request, memory and computational demands are greatly reduced. For example, **DeepSeek-V2** (236B parameters) activates just 21B parameters during inference. This enables PCs with AI SoC chips [6, 10, 58] to achieve nearly 20 tokens per second (TPS), or even twice that speed, which is more than sufficient for personal use. In contrast, dense models of similar capability (e.g., 70B parameters) typically reach only single-digit TPS on similar hardware.

Notably, the increasingly popular KTransformers [39] inference engine allows the complete DeepSeek-V3 model to run on a low-cost server equipped with a consumer GPU (costing approximately \$10,000), while still achieving nearly 20 TPS.

This efficiency makes MoE architectures suitable for local deployments and single-user scenarios, where hardware resources are often limited. By minimizing memory and computational overhead, MoE models can deliver high-quality inference performance without requiring expensive infrastructure.

2.3 Increasing Inference Speed

2.3.1 Overlapping Computation and Communication: Maximizing Throughput. Inference speed encompasses both system-wide maximum throughput and single-request latency. To maximize throughput, our model is architected from the outset to leverage dual micro-batch overlap [31, 79], intentionally overlapping communication latency with computation. As demonstrated in our online inference system and supported by open-source profiling data [31], we decouple the computation of MLA and MoE into two distinct stages. While one micro-batch executes a portion of MLA or MoE computation, the other micro-batch simultaneously performs the corresponding dispatch communication. Conversely, during the computation phase of the second micro-batch, the first micro-batch undergoes the combine communication step. This pipelined approach enables seamless overlap of all-to-all communication with ongoing computation, ensuring that the GPU remains fully utilized at all times. Moreover, in production, we adopt a prefill and decode

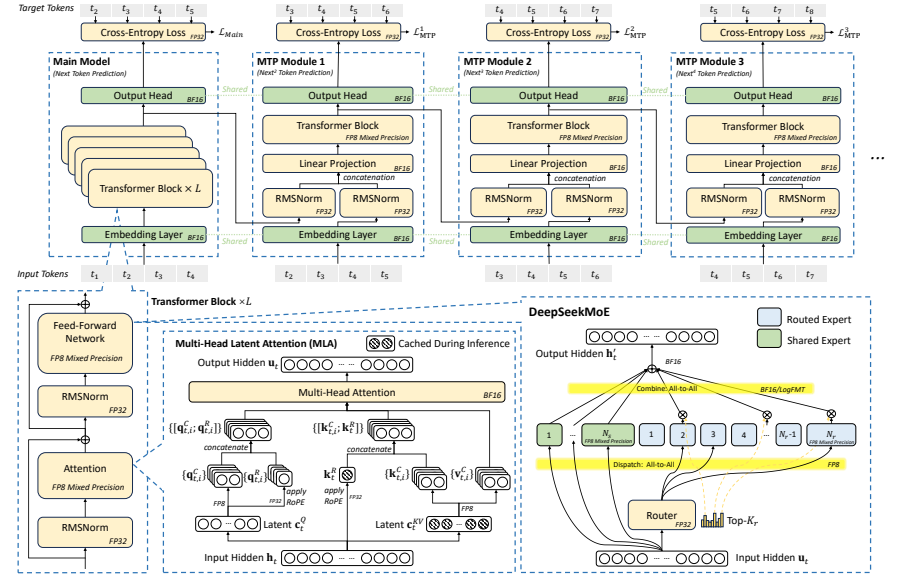


Figure 1

2 Design Principles for DeepSeek Models

DeepSeek-V3的发展体现了面向硬件的扩展大语言模型的方法，其中每个设计决策都经过精心调整，以适应硬件约束，从而优化性能和成本效益。

如图 1 所示，DeepSeek-V3 采用了 **DeepSeekMoE** [27] 和 **Multi-head Latent Attention (MLA)** [25] 架构，这些架构在 DeepSeek-V2 [25] 中已被证明是有效的。DeepSeekMoE 释放了 MoE 架构的潜力，而 MLA 则通过压缩键值 (KV) 缓存，大幅降低了内存消耗。

这些创新旨在解决扩展大语言模型中的三个核心挑战——**memory efficiency**、**cost-effectiveness** 和 **inference speed**——这些将在以下小节中详细探讨。

2.1 Memory Efficiency

大语言模型通常需要大量的内存资源，其内存需求每年增长超过 1000%。相比之下，高速内存（例如 HBM）容量的增长速度要慢得多，通常每年不到 50% [35]。虽然多节点并行是解决内存限制的可行方案，但从源头优化内存使用仍然是一个关键且有效的策略。

2.1.1 Low-Precision Models. 与使用 BF16 存储权重的模型相比，FP8 将内存消耗显著减少了一半，有效缓解了人工智能内存墙的挑战。关于低精度技术的详细讨论见第 3 节低精度驱动设计。

2.1.2 Reducing KV Cache with MLA. 对于 LLM 推理，用户请求通常涉及多轮对话。为了高效处理这些请求，先前请求的上下文被缓存在通常被称为 **KV cache** 的存储中。KV 缓存通过缓存已处理令牌的 **Key** 和 **Value** 向量来解决这一挑战，从而消除了为后续令牌重新计

disaggregation architecture [81], assigning large batch size prefill and latency-sensitive decode requests to different expert parallelism group sizes. This strategy ultimately maximizes system throughput under real-world service conditions.

2.3.2 Inference Speed Limits. This section focuses on the decode output speed of LLM services, typically measured in **Time Per Output Token (TPOT)**. TPOT is a critical metric for user experience, and it also directly impacts the responsiveness of reasoning models such as OpenAI’s o1/o3 and DeepSeek-R1, which rely on the inference length to enhance their intelligence.

For MoE models, achieving high inference speed relies on efficiently deploying expert parameters across computing devices. To achieve the fastest possible inference speed, each device should ideally perform computations for a single expert (or multiple devices should collaboratively compute a single expert if necessary). However, **Expert Parallelism (EP)** requires routing tokens to the appropriate devices, which involves all-to-all communication across the network. As a result, the upper limit of MoE inference speed is dictated by interconnection bandwidth.

Consider a system where each device holds one expert’s parameters and processes approximately 32 tokens at a time. This token count strikes a balance between compute-to-memory ratio and communication latency. And this token count ensures that each device processes an equal batch size during expert parallelism, allowing the communication time to be easily calculated.

For a system interconnected with CX7 400Gbps InfiniBand (IB) NICs, the time required for the two all-to-all communications in EP is calculated as follows:

Comm. Time = $(1\text{Byte} + 2\text{Bytes}) \times 32 \times 9 \times 7\text{K} / 50\text{GB/s} = 120.96\mu\text{s}$
Here, dispatch uses FP8 (1 byte), while combine uses BF16 (2 bytes), and the hidden size of each token is approximately 7K. The factor 9 indicates that each token is transferred to 8 routed experts and 1 shared expert. Network latency is not included in this calculation.

As discussed in Section 2.3.1, maximizing throughput necessitates the use of dual micro-batch overlap. In this strategy, our theoretical best-case analysis assumes that computation overhead is minimized, so the upper bound on performance is determined by communication latency. In practical inference workloads, however, request contexts are often much longer, and MLA computations typically dominate execution time. Thus, this analysis represents an idealized scenario under dual micro-batch overlap. Under this assumption, the total time per layer can be formulated as:

Total Time Per Layer = $2 \times 120.96\mu\text{s} = 241.92\mu\text{s}$
With 61 layers in DeepSeek-V3, the total inference time is:

Total Inference Time = $61 \times 241.92\mu\text{s} = 14.76\text{ms}$

Thus, the theoretical upper limit for this system is approximately **14.76 ms TPOT**, equivalent to **67 tokens per second**. However, in practice, factors such as communication overhead, latency, incomplete bandwidth utilization, and computational inefficiencies reduce this number.

By contrast, if a high-bandwidth interconnect like GB200 NVL72 (900GB/s unidirectional bandwidth across 72 GPUs) were used, the communication time per EP step drops to:

Comm. Time = $(1\text{Byte} + 2\text{Bytes}) \times 32 \times 9 \times 7\text{K} / 900\text{GB/s} = 6.72\mu\text{s}$

Assuming perfect overlap between computation and communication, this would yield a theoretical upper limit of **0.82 ms TPOT**, or approximately **1200 tokens per second**. However, this figure is purely theoretical and does not account for the substantial drop in GPU efficiency at small batch sizes; in real deployments, actual throughput will be significantly lower. Nonetheless, this calculation vividly illustrates the transformative potential of high-bandwidth scale-up networks in accelerating large-scale model inference.

While MoE models exhibit good scalability, achieving high inference speeds by increasing hardware resources alone is cost-prohibitive. Therefore, software and algorithms must also contribute to improving inference efficiency.

2.3.3 Multi-Token Prediction. Inspired by Gloeckle et al. [36], DeepSeek-V3 introduces a **Multi-Token Prediction (MTP)** framework, which simultaneously enhances model performance and improves inference speed. During inference, traditional autoregressive models generate one token at a decoding step, leading to sequential bottlenecks. MTP mitigates this issue by enabling the model to generate additional candidate tokens at a lower cost and verify them in parallel, similar to previous self-drafting-based speculative decoding approaches [14, 48]. This framework significantly accelerates inference without compromising accuracy.

As illustrated in the top part of Figure 1, each MTP module uses a single layer, which is much more lightweight than the full model, to predict additional tokens, enabling parallel verification of multiple candidate tokens. Although slightly hurting the throughput, this approach significantly improves the end-to-end generation latency. The real world practice data demonstrates that an MTP module achieves an acceptance rate of 80% to 90% for predicting the second subsequent token, which increases the generation TPS by 1.8x compared to the scenario without the MTP module.

Moreover, by predicting multiple tokens per step, MTP increases the inference batch size, which is crucial for boosting EP computational intensity and hardware utilization. Such algorithmic innovations are vital for fast and cost-effective inference in DeepSeek-V3.

2.3.4 High Inference Speed for Reasoning Models and Test-Time Scaling. Test-time scaling in LLMs, exemplified by OpenAI’s o1/o3 series [60, 61], has enabled significant advances in mathematical reasoning, programming, and general reasoning by dynamically adjusting computational resources during inference. Subsequent models—including DeepSeek-R1 [28], Claude-3.7 Sonnet [9], Gemini 2.5 Pro [38], Seed1.5-Thinking [68], and Qwen3 [72]—have adopted similar strategies and achieved notable improvements in these tasks.

For these reasoning models, high token output speed is of paramount importance. In reinforcement learning (RL) workflows—such as PPO [67], DPO [64] and GRPO [69]—the necessity to rapidly generate large numbers of samples makes inference throughput a critical bottleneck. Likewise, prolonged reasoning sequences can increase user wait times, reducing the practical usability of such models. As a result, optimizing inference speed through synergistic hardware and software innovations is indispensable for advancing the efficiency of reasoning models. However, effective strategies for accelerating inference and expediting RL training remain active areas of investigation, as discussed in Section 2.1.3. We encourage the broader community to collaboratively explore and develop novel solutions to these ongoing challenges.

Table 1: KV 缓存大小比较 (BF16 精度)：与其他使用 GQA 的模型相比，DeepSeek-V3 (MLA) 大幅减少了 KV 缓存大小。

Model	KV Cache Per Token	Multiplier
DeepSeek-V3 (MLA)	70.272 KB	1x
Qwen-2.5 72B (GQA)	327.680 KB	4.66x
LLaMA-3.1 405B (GQA)	516.096 KB	7.28x

算它们的需要。在每个推理步骤中，模型仅计算当前令牌的 Key 和 Value 向量，并通过将它们与历史缓存中的 Key-Value 对结合来执行注意力计算。这种增量计算将生成每个令牌的复杂度降低到 $O(N)$ ，使得在处理长序列或多轮输入时非常高效。然而，它也引入了内存瓶颈，因为计算从 GEMM 转向了 GEMV，而后的计算与内存比率要低得多。现代硬件提供数百 TFLOPS 的算力，GEMV 很快会受到内存带宽的限制，使得内存访问成为主要瓶颈。

为了解决这个瓶颈，我们采用了 **Multi-head Latent Attention (MLA)** [25]，它使用一个投影矩阵将所有注意力头的 KV 表示压缩成一个更小的潜在向量，该投影矩阵与模型联合训练。在推理过程中，只需要缓存潜在向量，与存储所有注意力头的 KV 缓存相比，显著降低了内存消耗。

除了 MLA 之外，还提出了其他几种方法来减小 KV 缓存的大小。这些方法非常有价值，并为内存高效注意力机制的进步提供了重要启示：

- **Shared KV (Grouped-Query Attention, GQA; Multi-Query Attention, MQA):** 不是为每个注意力头维护单独的 KV 对，而是多个头共享一组 KV 对，从而显著压缩 KV 存储。代表性方法包括 GQA [5] 和 MQA [70]。
- **Windowed KV:** 对于长序列，只保留缓存中的一个滑动窗口内的 KV 对，丢弃窗口外的结果。虽然这减少了存储，但损害了长上下文推理能力。代表性方法包括 Longformer [11] 及相关架构。
- **Quantized Compression:** 使用低比特表示 [40, 44, 52] 存储 KV 对，进一步减少内存使用。量化在最小化对模型性能影响的情况下实现了显著的压缩。

表 ?? 比较了 DeepSeek-V3、Qwen-2.5 72B [71] 和 LLaMA-3.1 405B [4] 每个令牌的 KV 缓存内存使用量。通过采

用 MLA，DeepSeek-V3 实现了 KV 缓存大小的显著减少，每个令牌仅需 70 KB，远低于 LLaMA-3.1 405B 的 516 KB 和 Qwen-2.5 72B 的 327 KB。这一减少凸显了 MLA 在压缩 KV 表示方面相较于基于 GQA 的方法的效率。这种显著降低内存消耗的能力使得 DeepSeek-V3 特别适合涉及长上下文处理和资源受限环境的场景，从而实现更具可扩展性和成本效益的推理。

2.1.3 Future Directions and Perspectives on Resource-Efficient Techniques. 虽然减小 KV 缓存大小是提高内存效率的一种有前景的方法，但基于 Transformer 的自回归解码所固有的二次复杂度仍然是一个巨大的挑战，尤其是在处理极长上下文时。近期的研究工作，如 Mamba-2 [21] 和 Lightning Attention[63]，探索了线性时间替代方案，为平衡计算成本和模型性能提供了新的可能性。此外，诸如稀疏注意力 [76] 等方法，旨在压缩并稀疏激活注意力键和值，代表了克服注意力相关计算挑战的另一种尝试。我们期待与更广泛的社区合作，共同推动该领域的突破。

2.2 Cost-Effectiveness of MoE Models

在稀疏计算方面，我们开发了 DeepSeekMoE，一种先进的 **Mixture of Experts (MoE)** 架构，如图 1 右下部分所示。

2.2.1 Reducing Computational Requirements for Training. MoE 架构的主要优势在于其能够显著降低训练成本。通过选择性地仅激活一部分专家参数，MoE 模型允许总参数量大幅增加，同时保持适度的计算需求。例如，DeepSeek-V2 拥有 2360 亿参数，但每个令牌仅激活 210 亿参数。类似地，DeepSeek-V3 扩展到 6710 亿参数——几乎是 V2 的三倍大小——而每个令牌的激活参数量仅为 370 亿。相比之下，Qwen2.5-72B 和 LLaMa3.1-405B 等稠密模型在训练时需要所有参数都处于激活状态。如表 2 所示，DeepSeek-V3 每个令牌的总计算成本约为 250 GFLOPS，而 720 亿参数的稠密模型需要 394 GFLOPS，4050 亿参数的稠密模型则需要 2448 GFLOPS。这表明 MoE 模型在消耗少一个数量级的计

2.4 Technique Validation Methodology

Each acceleration technique undergoes rigorous empirical validation to evaluate its accuracy impact, including MLA, FP8 mixed-precision computation, and network co-designed MoE gate routing. Given the prohibitive cost of exhaustive ablation on full-scale models, we adopt a hierarchical and resource-efficient validation pipeline. Each technique is first validated extensively on small-scale models, followed by minimal large-scale tuning, and finally integrated in a single, comprehensive training run. For instance, we first conducted fine-grained FP8 training ablation studies on both 16B and 230B DeepSeek-V2 models before final integration. Under these controlled settings, the relative accuracy loss compared to BF16 remains below 0.25%, attributable to our use of high-precision accumulation and fine-grained quantization strategies.

3 Low-Precision Driven Design

3.1 FP8 Mix-Precision Training

Quantization techniques such as GPTQ [32] and AWQ [51] have been widely used to reduce bit-widths to 8-bit, 4-bit, or even lower, significantly reducing memory requirements. However, these techniques are primarily applied during inference to save memory, rather than in the training phase. NVIDIA's Transformer Engine has supported FP8 mixed-precision training for some time, but prior to DeepSeek-V3, there were no open-source large models leveraging FP8 for training. Through deep collaboration between our infrastructure and algorithm teams, and after extensive experimentation and innovation, we developed an FP8-compatible training framework for MoE models. Figure 1 shows the computational components where FP8-precision forward and backward processes are utilized in the training pipeline. Fine-grained quantization is applied, i.e., tile-wise 1x128 quantization for activations and block-wise 128x128 quantization for model weights. Further technical details of our FP8 framework are documented in the DeepSeek-V3 technical report [26], and our fine-grained FP8 GEMM implementation has been open-sourced in DeepGEMM [78].

3.1.1 Limitations: While FP8 has great potential for accelerating training, several hardware limitations need to be addressed to fully exploit its capabilities:

- **FP8 Accumulation Precision:** FP8 uses constrained accumulation precision in Tensor Cores, affecting the stability for training large models, particularly on NVIDIA Hopper GPUs. After aligning 32 mantissa products by right-shifting based on the maximum exponent, the Tensor Core only maintains their highest 13 fraction bits for addition, and truncates bits exceeding this range. Addition results are accumulated to FP22 registers (1 sign bit, 8 exponent bits, and 13 mantissa bits)[77]. The term "FP22" follows the naming used in the cited work, and the FP8 precision issue on Hopper GPUs had also been observed across industry by early 2024.
- **Fine-Grained Quantization Challenges:** Fine-grained quantization such as tile-wise and block-wise quantization introduces large dequantization overhead in transporting the partial results from Tensor Cores to CUDA Cores for scaling factor multiplication. This incurs frequent data movements, reducing computational efficiency and complicating hardware utilization.

3.1.2 Suggestions: To address the limitations of existing hardware, we have the following suggestions for future designs:

- **Increased Accumulation Precision:** Hardware should improve the accumulation register precision to an appropriate value (e.g. FP32), or support a configurable accumulation precision, enabling a trade-off between performance and accuracy for different requirements of training and inference in various models.
- **Native Support for Fine-Grained Quantization:** Hardware should natively support fine-grained quantization, enabling Tensor Cores to receive scaling factors and implement matrix multiplication with group scaling. In this way, the whole partial sum accumulation and dequantization can be completed directly inside Tensor Cores until the final result is produced, avoiding frequent data movements to reduce dequantization overhead. A notable industrial implementation of this approach is NVIDIA Blackwell's support for **microscaling data format** [66], which exemplifies the practical benefits of native quantization at scale.

3.2 LogFMT: Communication Compression

In the current DeepSeek-V3 architecture, we employ low-precision compression for network communication. During EP parallelism, tokens are dispatched using fine-grained FP8 quantization, reducing communication volume by 50% compared to BF16. This significantly lowers communication time. While the combine stage still uses higher precision (e.g., BF16) due to accuracy requirements, we are actively testing FP8, custom precision formats (e.g., E5M6) and mixing FP8-BF16 for further reductions.

Besides these traditional floating point formats, we also tried a new data type, named **Logarithmic Floating-Point Formats (LogFMT-nBit)**, where n is the number of bits with the leading 1 bit as the sign bit S . By mapping the activations from the original Linear space to the Log space, the distribution of the activations is more uniform. To be specific, given a tile of elements, $[x_1, \dots, x_m]$, which is 1x128 in our implementation, we take the absolute values and compute the logarithm of all the elements, and find the minimum $min = \log(abs(x_i))$ and maximum $max = \log(abs(x_j))$. The minimum is encoded as $S.00 \dots 01$ and the maximum is encoded as $S.11 \dots 11$, with an interval representing $Step = \frac{max-min}{2^n-1}$. Zero values are represented by $S.00 \dots 00$, specially. The left values are rounded to the nearest integer K multiples of $Step$. The decoding process is simple by combining the sign bit and $exp^{min+Step \times (K-1)}$.

By locally calculating the min and $Step$, this data type supports dynamic representation range for different blocks, covering larger ranges or providing more precision, compared to static floating point formats. Besides, we find it is important to round in the original Linear space, instead of the Log space, for the unbiased activation quantization. We also constrain the min to be larger than $max - \log(2^{32})$, which means that the max representation range is similar to E5, a floating point with 5 exponents. We validate our LogFMT-nBit on dense language models with around 7 billion parameters, by quantifying the output of the residual branch to simulate the combine stage in MoE models. When setting $n = 8$, sharing the same bits with FP8, the LogFMT-8Bit shows superior training accuracy compared to E4M3 or E5M2. After increasing the n to 10 bits, we find it's similar to the BF16 combine stage.

Table 2: MoE 与稠密模型训练计算成本对比: 测量每个词元的计算成本, 假设序列长度为 4096。

Model	Size	Training Cost
DeepSeek-V2 MoE	236B	155 GFLOPS/Token
DeepSeek-V3 MoE	671B	250 GFLOPS/Token
Qwen-72B Dense	72B	394 GFLOPS/Token
LLaMa-405B Dense	405B	2448 GFLOPS/Token

算资源的同时, 实现了与稠密模型相当甚至更优的性能。

2.2.2 Advantages for Personal Use and On-Premises Deployment. 在一个个性化 LLM 智能体 [53] 变得无处不在的未来, MoE 模型在单次请求场景中提供了独特的优势。因为每次请求仅激活一部分参数, 内存和计算需求被大大降低。例如, **DeepSeek-V2** (2360 亿参数) 在推理过程中仅激活 210 亿参数。这使得配备 AI SoC 芯片 [6, 10, 58] 的个人电脑能够实现近每秒 20 个词元 (TPS) 的速度, 甚至可以达到两倍于此, 这对于个人使用来说绰绰有余。相比之下, 能力相似 (例如, 700 亿参数) 的稠密模型在类似硬件上通常只能达到个位数的 TPS。

这种效率使得 MoE 架构适用于本地部署和单用户场景, 这些场景的硬件资源通常有限。通过最小化内存和计算开销, MoE 模型无需昂贵的基础设施即可提供高质量的推理性能。

2.3 Increasing Inference Speed

2.3.1 Overlapping Computation and Communication: Maximizing Throughput. 推理速度既包括系统范围内的最大吞吐量, 也包括单次请求的延迟。为了最大化吞吐量, 我们的模型从一开始就设计为利用双微批次重叠 [31, 79], 有意将通信延迟与计算重叠。正如我们的在线推理系统所展示的, 并由开源性能分析数据 [31] 所支持, 我们将 MLA 和 MoE 的计算解耦为两个不同的阶段。当一个微批次执行一部分 MLA 或 MoE 计算时, 另一个微批次同时执行相应的分发通信。反之, 在第二个微批次的计算阶段, 第一个微批次则进行组合通信步骤。这种流水线方法使得全对全通信能够与正在进行的计算无缝重叠, 确保 GPU 始终保持完全利

用。此外, 在生产中, 我们采用预填充和解码分离架构 [81], 将大批次大小的预填充请求和对延迟敏感的解码请求分配给不同规模的专家并行组。这一策略最终在实际服务条件下最大化系统吞吐量。

2.3.2 Inference Speed Limits. 本节重点关注 LLM 服务的解码输出速度, 通常以 **Time Per Output Token (TPOT)** 衡量。TPOT 是用户体验的关键指标, 它也直接影响诸如 OpenAI 的 o1/o3 和 DeepSeek-R1 这类依赖推理长度来提升智能的推理模型的响应速度。

对于 MoE 模型, 实现高推理速度依赖于将专家参数高效地部署在计算设备上。为了达到尽可能快的推理速度, 理想情况下每个设备应执行单个专家的计算 (或者, 如有必要, 多个设备应协作计算单个专家)。然而, $\ll\text{LEX}_{MD_3}\ggg$ all-to-all MoE

考虑一个系统, 其中每个设备持有一个专家的参数, 并每次处理大约 32 个令牌。这个令牌数量在计算与内存之比和通信延迟之间取得了平衡。并且这个令牌数量确保了在专家并行期间每个设备处理相等的批次大小, 使得通信时间易于计算。

对于一个使用 CX7 400Gbps InfiniBand (IB) 网卡互连的系统, EP 中两次 all-to-all 通信所需的时间计算如下:

$$\text{Comm. Time} = (1\text{Byte}+2\text{Bytes}) \times 32 \times 9 \times 7\text{K} / 50\text{GB/s} = 120.96\mu\text{s}$$

这里, 分发使用 FP8 (1 字节), 而合并使用 BF16 (2 字节), 每个令牌的隐藏大小约为 7K。因子 9 表示每个令牌被传输到 8 个路由专家和 1 个共享专家。此计算未包含网络延迟。

如第 2.3.1 节所述, 最大化吞吐量需要使用双微批次重叠。在此策略中, 我们的理论最佳情况分析假设计算开销最小化, 因此性能上限由通信延迟决定。然而, 在实际推理工作负载中, 请求上下文通常要长得多, 并且 MLA 计算通常主导执行时间。因此, 此分析代表了双微批次重叠下的理想化场景。在此假设下, 每层的总时间可以表示为:

3.2.1 Limitations: The initial purpose of using LogFMT is to apply it to activations during transmission or near activation functions, as it offers higher precision than FP8 with the same bit width. However, subsequent computations require reconversion to BF16 or FP8 to accommodate the Hopper GPU tensor cores' data type. Due to insufficient GPU bandwidth for log/exp operations and excessive register pressure during encode/decode, if encode/decode operations are fused with all-to-all communication, the overhead can be substantial (50%~100%). Therefore, although experimental results validate the effectiveness of this format, we do not employ it eventually.

3.2.2 Suggestions: Providing native support for compression and decompression units tailored to FP8 or custom precision formats represents a viable approach for future hardware. This could help minimize bandwidth requirements and streamline communication pipelines. The reduced communication overhead is particularly helpful in bandwidth-intensive tasks like MoE training.

4 Interconnection Driven Design

4.1 Current Hardware Architecture

The NVIDIA H800 GPU SXM architecture we currently use, illustrated in Figure 2, is built on the Hopper architecture, similar to the H100 GPU. However, it features reduced FP64 computational performance and NVLink bandwidth for regulatory compliance. Specifically, the NVLink bandwidth in H800 SXM nodes is reduced from 900 GB/s to 400 GB/s. This significant reduction in intra-node scale-up bandwidth presents a challenge for high-performance workloads. To compensate, each node is equipped with eight 400G InfiniBand (IB) CX7 NICs, enhancing scale-out capabilities to mitigate the bandwidth deficit.

To address these hardware constraints, the DeepSeek-V3 model incorporates several design considerations that align with the hardware's strengths and limitations.

4.2 Hardware-Aware Parallelism

To align with the constraints of the H800 architecture, the following parallelism strategies were considered to optimize the performance of DeepSeek-V3:

- **Avoidance of Tensor Parallelism (TP):** Tensor Parallelism is avoided during training due to its inefficiency under limited NVLink bandwidth. However, during inference, TP can still be selectively used to improve TTFT and TPOT performance.
- **Enhanced Pipeline Parallelism (PP):** DualPipe [29] is employed to overlap attention and MoE computation with MoE communication. This also reduces pipeline bubbles and balances memory usage across GPUs, improving overall throughput. Additional details are available in the technical report [26].
- **Accelerated Expert Parallelism (EP):** With eight 400Gbps InfiniBand (IB) NICs, the system achieves all-to-all communication at speeds exceeding 40GB/s. Notably, our all-to-all EP implementation, DeepEP [79], is open-sourced, enabling highly efficient expert parallelism as discussed in the following subsection.

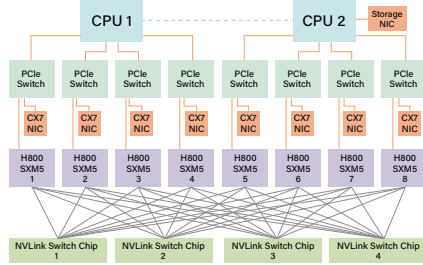


Figure 2: H800 node interconnection.

4.3 Model Co-Design: Node-Limited Routing

The bandwidth disparity between scale-up (intra-node) and scale-out (inter-node) communication in the H800 architecture is approximately 4:1. Specifically, NVLink provides 200GB/s bandwidth (of which about 160GB/s can actually be achieved), while each 400Gbps IB NIC delivers only 50GB/s bandwidth (we consider small message size and latency influence, use 40GB/s for effective bandwidth). To balance and fully utilize the higher intra-node bandwidth, the model architecture is co-designed with hardware, particularly in the **TopK Expert Selection Strategy**.

Consider a setup with 8 nodes (64 GPUs in total) and 256 routed experts (4 experts per GPU). For DeepSeek-V3, each token is routed to one shared expert and 8 routed experts. If its 8 target experts are distributed across all 8 nodes, the communication time over IB would be $8t$, where t represents the time to send one token over IB. However, by leveraging the higher NVLink bandwidth, tokens routed to the same node can be sent once over IB and then forwarded via NVLink to other intra-node GPUs. The NVLink forwarding enables deduplication of the IB traffic. When the target experts for a given token are distributed across M nodes, the deduplicated IB communication cost will be reduced to Mt ($M < 8$).

Since the IB traffic depends on only M , DeepSeek-V3 introduces a **Node-Limited Routing** for the TopK expert selection strategy. Specifically, we group 256 routed experts into 8 groups, with 32 experts per group, and deploy each group on a single node. On top of this deployment, we algorithmically ensure that each token will be routed to up to 4 nodes. This approach mitigates the bottleneck of IB communication and enhances the effective communication bandwidth during training.

4.4 Scale-Up and Scale-Out Convergence

4.4.1 Limitations of Current Implementations. While the Node-Limited Routing strategy reduces communication bandwidth requirements, it complicates communication pipeline kernel implementations due to the disparity in bandwidth between intra-node (NVLink) and inter-node (IB) interconnects. In practice, GPU Streaming Multiprocessors (SM) threads are used for both network message handling (e.g., filling QPs and WQEs) and data forwarding over NVLink, consuming computational resources. For example, during training, up to 20 of the SMs on the H800 GPU are allocated for

$$\text{Total Time Per Layer} = 2 \times 120.96\mu\text{s} = 241.92\mu\text{s}$$

对于具有 61 层的 DeepSeek-V3，总推理时间为：

$$\text{Total Inference Time} = 61 \times 241.92\mu\text{s} = 14.76\text{ms}$$

因此，该系统的理论上限约为 **14.76 ms TPOT**，相当于 **67 tokens per second**。然而，在实践中，诸如通信开销、延迟、带宽利用不完全以及计算效率低下等因素会降低这个数值。

相比之下，如果使用像 GB200 NVL72 这样的高带宽互连（跨 72 个 GPU 的 900GB/s 单向带宽），每个 EP 步骤的通信时间将降至：

$$\text{Comm. Time} = (1\text{Byte} + 2\text{Bytes}) \times 32 \times 9 \times 7\text{K} / 900\text{GB/s} = 6.72\mu\text{s}$$

假设计算与通信之间完全重叠，这将产生 **0.82 ms TPOT** 的理论上限，或大约 **1200 tokens per second**。然而，这个数字纯粹是理论上的，并未考虑在小批次大小时 GPU 效率的显著下降；在实际部署中，实际吞吐量将显著降低。尽管如此，这个计算生动地说明了高带宽纵向扩展网络在加速大规模模型推理方面的变革潜力。

Expert Parallelism (EP) 虽然 MoE 模型展现出良好的可扩展性，但仅通过增加硬件资源来实现高推理速度成本过高。因此，软件和算法也必须为提升推理效率做出贡献。

2.3.3 Multi-Token Prediction. 受 Gloeckle et al. [36] 启发，DeepSeek-V3 引入了 **Multi-Token Prediction (MTP)** 框架，该框架同时提升了模型性能并改进了推理速度。在推理过程中，传统的自回归模型在每个解码步骤生成一个词元，导致了顺序瓶颈。MTP 通过使模型能够以较低成本生成额外的候选词元并行验证它们来缓解此问题，类似于先前的基于自草稿的推测解码方法 [14, 48]。该框架在不影响准确性的情况下显著加速了推理。

如图 1 顶部所示，每个 MTP 模块使用一个单层（比完整模型轻量得多）来预测额外的词元，从而实现多个候选词元的并行验证。虽然略微影响了吞吐量，

但这种方法显著改善了端到端的生成延迟。真实世界实践数据表明，MTP 模块在预测第二个后续词元时达到了 80% 到 90% 的接受率，与没有 MTP 模块的场景相比，生成 TPS 提高了 1.8 倍。

此外，通过每个步骤预测多个词元，MTP 增加了推理批次大小，这对于提升 EP 计算强度和硬件利用率至关重要。此类算法创新对于 DeepSeek-V3 实现快速且经济高效的推理至关重要。

2.3.4 High Inference Speed for Reasoning Models and Test-Time Scaling. 以 OpenAI 的 o1/o3 系列 [60, 61] 为例，LLM 中的测试时缩放通过在推理过程中动态调整计算资源，在数学推理、编程和一般推理方面实现了显著进步。后续模型——包括 DeepSeek-R1 [28]、Claude-3.7 Sonnet [9]、Gemini 2.5 Pro [38]、Seed1.5-Thinking [68] 和 Qwen3 [72]——都采用了类似的策略，并在这些任务上取得了显著改进。

对于这些推理模型，高词元输出速度至关重要。在强化学习 (RL) 工作流程中——例如 PPO [67]、DPO [64] 和 GRPO [69]——需要快速生成大量样本，这使得推理吞吐量成为一个关键瓶颈。同样，过长的推理序列会增加用户等待时间，降低此类模型的实际可用性。因此，通过协同的硬件和软件创新来优化推理速度，对于提升推理模型的效率是不可或缺的。然而，加速推理和加快 RL 训练的有效策略仍然是活跃的研究领域，如第 2.1.3 节所述。我们鼓励更广泛的社区共同探索和开发应对这些持续挑战的新颖解决方案。

2.4 Technique Validation Methodology

每种加速技术都经过严格的实证验证以评估其对精度的影响，包括 MLA、FP8 混合精度计算以及网络协同设计的 MoE 门控路由。鉴于在完整规模模型上进行详尽消融实验的成本过高，我们采用了一种分层且资源高效的验证流程。每项技术首先在小型模型上进行广泛验证，随后进行最小化的大规模调优，最后整合到一次全面、综合的训练运行中。例如，在最终集成之前，我们首先在 16B 和 230B 的 DeepSeek-V2 模型上进

communication-related operations, leaving fewer resources available for actual computation. To maximize throughput in online inference, we perform EP all-to-all communication entirely through NIC RDMA, avoiding SM resource contention and improving compute efficiency. This highlights the advantage of RDMA’s asynchronous communication model in overlapping computation and communication.

The following are key tasks currently performed by SMs during EP communication, particularly for the combine stage’s reduce operations and data type conversions. Offloading these tasks to dedicated communication hardware could free up SMs for computation kernels, significantly improving overall efficiency:

- **Forwarding Data:** Aggregating IB traffic destined for multiple GPUs within the same node between the IB and NVLink domains.
- **Data Transport:** Moving data between RDMA buffers (registered GPU memory regions) and input/output buffers.
- **Reduce Operations:** Executing reduce operations required for EP all-to-all combine communications.
- **Managing Memory Layouts:** Handling fine-grained memory layouts for chunked data transfers across the IB and NVLink domains.
- **Data Type Cast:** Converting data type before and after all-to-all communications.
- **Unified Network Adapter:** Design NICs (Network Interface Cards) or I/O Dies that are connected to unified scale-up and scale-out networks. These adapters should also support basic switch functionality, such as forwarding packets from the scale-out network to specific GPUs within the scale-up network. This could be achieved using a single LID (Local Identifier) or IP address with policy-based routing.
- **Dedicated Communication Co-Processor:** Introduce a dedicated co-processor or programmable component—such as an I/O die—for handling network traffic. This component should offload packet processing from GPU SMs, provide hardware-accelerated memory copy for efficient buffer management, and, crucially, accelerate memory load/store operations in a manner similar to TMA (Tensor Memory Accelerator), thereby saturating bandwidth with minimal resource consumption.
- **Flexible Forwarding, Broadcast and Reduce Mechanisms:** Hardware should support flexible forwarding, broadcast operations (for EP dispatch), and reduce operations (for EP combine) across scale-up and scale-out networks—mirroring our current GPU SM-based implementation. This would not only improve effective bandwidth but also reduce the computational complexity of network-specific operations.
- **Hardware Synchronization Primitives:** Provide fine-grained hardware synchronization instructions to handle memory consistency issues or out-of-order packet arrivals at the hardware level. This would eliminate the need for software-based synchronization mechanisms like RDMA completion events, which introduce extra latency and increase programming complexity. Memory-semantic communication with an acquire/release mechanism is a promising implementation.
- **Dynamic NVLink/PCIe Traffic Prioritization:** Hardware should support dynamic prioritization of traffic based on its type. For example, traffic related to EP, TP, and KV cache transfers should be assigned different priorities to maximize interconnect

efficiency. For PCIe, exposing the traffic class (TC) to user-level programming would suffice.

- **I/O Die Chiplet Integration:** Integrating NICs directly into the I/O die and connecting them to the compute die in the same package, rather than through conventional PCIe, would substantially reduce communication latency and alleviate PCIe bandwidth contention.
- **CPU-GPU Interconnects within the Scale-Up Domain:** To further optimize intra-node communication, CPUs and GPUs should be interconnected using NVLink or similar dedicated high-bandwidth fabrics, rather than relying solely on PCIe. Similar to the benefits provided by integrating NICs into the I/O die, this approach can significantly improve scenarios such as offloading parameters or KV cache between GPU and CPU memory during training and inference.
- **Subset of Multi-Rail Fat-Tree (MRFT):** The MPFT topology constitutes a specific subset of the broader MRFT architecture. As a result, existing optimizations developed by NVIDIA and NCCL for Multi-Rail networks can be seamlessly leveraged within Multi-Plane network deployments. Furthermore, NCCL’s support for PXN [54] technology addresses the inherent challenge of inter-plane isolation, enabling efficient communication even when direct interconnectivity between planes is absent.
- **Cost Efficiency:** As shown in Table 3, the multi-plane network enables over 10k endpoints using a two-layer fat-tree (FT2) topology, significantly reducing network costs compared to a three-layer fat tree (FT3). The cost per endpoint is even slightly more competitive than the cost-efficient Slim Fly (SF) topology [12].
- **Traffic Isolation:** Each plane operates independently, ensuring that congestion in one plane does not affect others. This isolation improves overall network stability and prevents cascading performance degradation.
- **Latency Reduction:** The two-layer topology achieves lower latency than three-layer fat trees, as demonstrated in our experiments. This makes it particularly suitable for latency-sensitive applications such as MoE-based training and inference.
- **Robustness:** As shown in Figure 4, multi-port NICs provide multiple uplinks, so single-port failures do not disrupt connectivity and rapid, transparent fault recovery is possible.
- **Cost:** IB hardware is significantly more expensive than RoCE solutions, which limits its widespread adoption.
- **Scalability:** IB switches typically support only 64 ports per switch, compared to the 128 ports commonly found in RoCE switches. This restricts the scalability of IB-based clusters, particularly for large-scale deployments.

4.4.2 Suggestions: To address these inefficiencies, we strongly recommend that future hardware should integrate intra-node (scale-up) and inter-node (scale-out) communication into a unified framework. By incorporating dedicated co-processors for network traffic management and seamless forwarding between NVLink and IB domains, such designs can reduce software complexity and maximize bandwidth utilization. For example, node-limited routing strategies employed in DeepSeek-V3 can be further optimized with hardware support for dynamic traffic deduplication.

We also recognize emerging interconnect protocols such as the Ultra Ethernet Consortium (UEC) [17, 18], Ultra Accelerator Link (UALink) [16], both of which are poised to drive advancements in

行了细粒度的 FP8 训练消融研究。在这些受控设置下，与 BF16 相比，相对精度损失保持在 0.25% 以下，这归因于我们使用了高精度累加和细粒度的量化策略。

3 Low-Precision Driven Design

3.1 FP8 Mix-Precision Training

诸如 GPTQ [32] 和 AWQ [51] 等量化技术已被广泛用于将位宽降低至 8 位、4 位甚至更低，从而显著减少内存需求。然而，这些技术主要应用于推理阶段以节省内存，而非训练阶段。NVIDIA 的 Transformer Engine 已支持 FP8 混合精度训练一段时间，但在 DeepSeek-V3 之前，尚无开源大模型利用 FP8 进行训练。通过我们基础设施团队与算法团队的深度协作，并经过大量实验和创新，我们为 MoE 模型开发了一个兼容 FP8 的训练框架。图 1 展示了训练流程中利用 FP8 精度前向和反向过程的计算组件。我们应用了细粒度量化，即对激活值采用分片 (tile-wise) 1x128 量化，对模型权重采用分块 (block-wise) 128x128 量化。我们 FP8 框架的进一步技术细节记录在 DeepSeek-V3 技术报告 [26] 中，并且我们的细粒度 FP8 GEMM 实现已在 DeepGEMM [78] 中开源。This section focuses on how to effectively utilize FP8 hardware capabilities, the challenges we encountered, and new practices derived from our experience.

3.1.1 Limitations: 尽管 FP8 在加速训练方面具有巨大潜力，但要充分发挥其能力，仍需解决若干硬件限制：

- **FP8 Accumulation Precision:** FP8 在 Tensor Core 中使用受限的累加精度，这影响了训练大模型的稳定性，尤其是在 NVIDIA Hopper GPU 上。在根据最大指数通过右移对齐 32 个尾数乘积后，Tensor Core 仅保留其最高的 13 个小数位用于加法，并截断超出此范围的位。加法结果累加到 FP22 寄存器（1 个符号位，8 个指数位和 13 个尾数位）[77]。术语“FP22”遵循所引用工作中的命名方式，并且 Hopper GPU 上的 FP8 精度问题在 2024 年初也已在整个业界被观察到。

- **Fine-Grained Quantization Challenges:** 诸如分片量化和分块量化等细粒度量化，在将部分结果从 Tensor Core 传输到 CUDA Core 进行缩放因子乘法时，引入了较大的反量化开销。这导致了频繁的数据移动，降低了计算效率并使硬件利用复杂化。

3.1.2 Suggestions: 针对现有硬件的局限性，我们对未来设计提出以下建议：

- **Increased Accumulation Precision:** 硬件应将累加寄存器的精度提升至适当值（例如 FP32），或支持可配置的累加精度，从而针对不同模型在训练和推理中的不同需求，在性能和精度之间进行权衡。
- **Native Support for Fine-Grained Quantization:** 硬件应原生支持细粒度量化，使张量核心能够接收缩放因子并实现带分组缩放的矩阵乘法。这样，整个部分和累加与反量化过程可以直接在张量核心内部完成，直至产生最终结果，从而避免频繁的数据移动以减少反量化开销。该方法一个值得注意的工业实现是 NVIDIA Blackwell 对 **microscaling data format** [66] 的支持，这例证了大规模原生量化的实际效益。

3.2 LogFMT: Communication Compression

在当前的 DeepSeek-V3 架构中，我们采用低精度压缩进行网络通信。在 EP 并行期间，令牌使用细粒度 FP8 量化进行分发，与 BF16 相比，通信量减少了 50%。这显著降低了通信时间。虽然 combine 阶段由于精度要求仍使用更高精度（例如 BF16），但我们正在积极测试 FP8、自定义精度格式（例如 E5M6）以及混合 FP8-BF16 以进一步减少通信量。除了这些传统的浮点格式，我们还尝试了一种新的数据类型，名为 **Logarithmic Floating-Point Formats (LogFMT-nBit)**，其中 n 是位数，最高位为符号位 S 。通过将激活值从原始的 Linear 空间映射到 Log 空间，激活值的分布更加均匀。具体来说，给定一个元素块 $[x_1, \dots, x_m]$ （在我们的实现中是 1x128），我们取所有元素的绝对值并计算其对数，然后找到最小值 $\min = \log(\text{abs}(x_i))$ 和最

scale-up and scale-out communication. More recently, Unified Bus (UB) [49] has introduced a novel approach to scale-up and scale-out convergence. Section 6 further explores several technical innovations proposed by UEC and UALink. However, in this section, our primary focus is on achieving scale-up and scale-out convergence at the programming framework level:

By implementing these recommendations, future hardware designs can significantly enhance the efficiency of large-scale distributed AI systems while simplifying software development.

4.5 Bandwidth Contention and Latency

4.5.1 Limitations: Besides, current hardware lacks the flexibility to dynamically allocate bandwidth between different types of traffic on NVLink and PCIe. For example, during inference, transferring KV cache data from CPU memory to GPU can consume tens of GB/s, saturating PCIe bandwidth. If the GPU simultaneously uses IB for EP communication, this contention between KV cache transfers and EP communication can degrade overall performance and cause latency spikes.

4.5.2 Suggestions:

5 Large Scale Network Driven Design

5.1 Network Co-Design: Multi-Plane Fat-Tree

During the training of DeepSeek-V3, we deployed a **Multi-Plane Fat-Tree (MPFT)** scale-out network, as shown in Figure 3. Each node is equipped with eight GPUs and eight IB NICs, with each GPU–NIC pair assigned to a distinct network plane. Additionally, each node has a 400 Gbps Ethernet RoCE NIC connected to a separate storage network plane for accessing the 3FS [30] distributed file system. In the scale-out network, we used 64-port 400G IB switches, enabling the topology theoretically supports up to 16,384 GPUs while retaining the cost and latency advantages of a two-layer network. However, due to policy and regulatory constraints, just over two thousand GPUs were ultimately deployed.

Furthermore, due to the current limitations of IB ConnectX-7, our deployed MPFT network does not fully realize the envisioned architecture. Ideally, as depicted in Figure 4, each NIC would feature multiple physical ports, each connected to a separate network

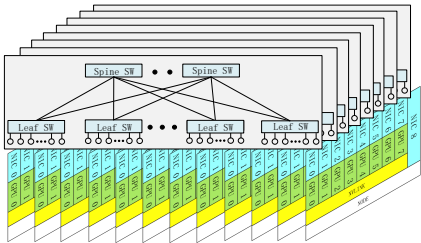


Figure 3: Eight-plane two-layer fat-tree scale-out network: Each GPU and IB NIC pair belongs to one network plane. Cross-plane traffic must use another NIC and PCIe or NVLink for intra-node forwarding.

Table 3: Network topology comparison. Cost estimates are derived from the methodology in the Slim Fly (SF) paper [12]. DF denotes the canonical dragonfly topology [22, 46, 65].

Metric	FT2	MPFT	FT3	SF	DF
Endpoints	2,048	16,384	65,536	32,928	261,632
Switches	96	768	5,120	1,568	16,352
Links	2,048	16,384	131,072	32,928	384,272
Cost [M\$]	9	72	491	146	1,522
Cost/Endpoint [k\$]	4.39	4.39	7.5	4.4	5.8

plane, yet collectively exposed as a single logical interface to the user through port bonding. From a user perspective, a single Queue Pair (QP) could seamlessly transmit and receive messages across all available ports, akin to packet spraying. As a consequence, packets originating from the same QP may traverse distinct network paths and arrive at the receiver out of order, thereby necessitating native support for out-of-order placement within the NIC to guarantee message consistency and preserve the correct ordering semantics. For example, InfiniBand ConnectX-8 natively supports four plane. It would be advantageous for future NICs to fully support advanced multi-plane capabilities, allowing two-tier fat-tree networks to scale effectively to much larger AI clusters. Overall, the multi-plane architecture offers significant advantages in fault isolation, robustness, load balancing, and large-scale system scalability.

5.1.1 Advantages of Multi-Plane Fat-Tree Network.

It is important to note that, due to current 400G NDR InfiniBand limitations, cross-plane communication requires intra-node forwarding, which introduces additional latency during inference. If future hardware can achieve scale-up and scale-out network convergence as discussed earlier, this latency can be significantly reduced, further enhancing the viability of multi-plane networks.

5.1.2 Performance Analysis. To verify the effectiveness of the Multi-Plane Network design, we conducted real-world experiments on our cluster, modifying the cluster’s network topology to compare the performance of the **Multi-Plane Two-Layer Fat Tree (MPFT)** and the **Single-Plane Multi-Rail Fat Tree (MRFT)**. Below are the key findings from our experiments:

1. **All-to-All Communication and EP Scenarios:** As illustrated in Figure 5, the all-to-all performance of the multi-plane network is very similar to that of the single-plane multi-rail network. This performance parity can be attributed to NCCL’s PXN [54] mechanism, which optimizes traffic forwarding via NVLink in multi-rail topologies. The multi-plane topology also benefits from this mechanism. As shown in Figure 6, the results of all-to-all communication tests conducted on 16 GPUs reveal negligible differences in latency between the MPFT and MRFT topologies.

To evaluate MPFT’s performance of all-to-all communication in practical training scenarios, we tested the EP communication patterns commonly used during training. As shown in Figure 7, each GPU achieves a high bandwidth exceeding 40GB/s in a multi-plane network, providing reliable performance that meets the demands of training.

2. **Training Throughput for DeepSeek-V3 Model:** We also compare the training metrics of the DeepSeek-V3 model between

大值 $\max = \log(\text{abs}(x_j))$ 。最小值被编码为 $S.00 \dots 01$ ，最大值被编码为 $S.11 \dots 11$ ，区间代表 $\text{Step} = \frac{\max - \min}{2^{n-1} - 2}$ 。零值由 $S.00 \dots 00$ 特殊表示。其余值被舍入到最接近的整数 K 乘以 Step 的倍数。解码过程很简单，只需结合符号位和 $\exp^{\min + \text{Step} \times (K-1)}$ 。

通过在本地计算 $\min$ 和 Step ，与静态浮点格式相比，这种数据类型支持针对不同块的动态表示范围，可以覆盖更大范围或提供更高精度。此外，我们发现对于无偏激活量化，在原始的 Linear 空间中进行舍入，而不是在 Log 空间中，是很重要的。我们还约束 $\min$ 大于 $\max - \log(2^{32})$ ，这意味着最大表示范围类似于 E5（一种具有 5 位指数的浮点数）。我们通过量化残差分支的输出来模拟 MoE 模型中的 combine 阶段，从而在约 70 亿参数的稠密语言模型上验证了我们的 LogFMT-nBit。当设置 $n = 8$ （与 FP8 位数相同）时，与 E4M3 或 E5M2 相比，LogFMT-8Bit 显示出更优的训练精度。将 n 增加到 10 位后，我们发现其效果类似于 BF16 的 combine 阶段。This ideas can be extended to other numerical space. [For something we tried atan2, reverse space etc.] Future hardware should incorporate support for Logarithmic Floating-Point Formats (LogFMT), which can extend the dynamic range of low-precision computations. This would mitigate precision loss while maintaining the performance benefits of low-precision arithmetic.

3.2.1 Limitations: 使用 LogFMT 的初衷是将其应用于传输过程中或激活函数附近的激活值，因为它在相同位宽下比 FP8 提供更高的精度。然而，后续计算需要重新转换为 BF16 或 FP8，以适应 Hopper GPU 张量核心的数据类型。由于 GPU 带宽不足以支持对数/指数运算，并且在编码/解码过程中寄存器压力过大，如果将编码/解码操作与 all-to-all 通信融合，开销可能相当大（50%~100%）。因此，尽管实验结果验证了这种格式的有效性，我们最终并未采用它。

3.2.2 Suggestions: 为 FP8 或自定义精度格式量身定制压缩和解压缩单元提供原生支持，是未来硬件的一种可行方案。这有助于最小化带宽需求并简化通信流水

线。减少的通信开销对于像 MoE 训练这样的带宽密集型任务尤其有帮助。

4 Interconnection Driven Design

4.1 Current Hardware Architecture

Architecture

我们当前使用的 NVIDIA H800 GPU SXM 架构,如图 2 所示,基于 Hopper 架构构建,与 H100 GPU 类似。然而,为符合监管要求,其 FP64 计算性能和 NVLink 带宽有所降低。具体而言, H800 SXM 节点中的 NVLink 带宽从 900 GB/s 降至 400 GB/s。节点内纵向扩展带宽的显著降低对高性能工作负载构成了挑战。为弥补这一点,每个节点配备了八个 400G Infiniband (IB) CX7 网卡,增强了横向扩展能力以缓解带宽不足。

为了应对这些硬件限制, DeepSeek-V3 模型融合了多项设计考量,旨在与硬件的优势和局限相匹配。

4.2 Hardware-Aware Parallelism

为了与 H800 架构的约束条件对齐,我们考虑了以下并行策略来优化 DeepSeek-V3 的性能:

- **Avoidance of Tensor Parallelism (TP):** 在训练期间避免使用张量并行, 因为其在有限的 NVLink 带宽下效率不高。然而, 在推理期间, 仍可选择性使用 TP 来提升 TTFT 和 TPOT 性能。
- **Enhanced Pipeline Parallelism (PP):** 采用 DualPipe [29] 来重叠注意力计算、MoE 计算与 MoE 通信。这同时减少了流水线气泡, 并平衡了 GPU 间的内存使用, 从而提高了整体吞吐量。更多细节可参见技术报告 [26]。
- **Accelerated Expert Parallelism (EP):** 借助八个 400Gbps InfiniBand (IB) 网卡, 系统能够以超过 40GB/s 的速度进行全对全通信。值得注意的是, 我们实现的全对全 EP, 即 DeepEP [79], 已开源, 使得如后续小节所述的高效专家并行成为可能。

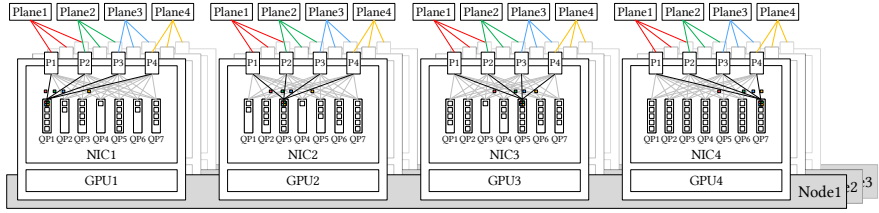


Figure 4: Ideal Multi-Plane Network: Each NIC is equipped with multiple physical ports, each connected to a distinct network plane. A single queue pair (QP) can simultaneously utilize all available ports for transmitting and receiving packets, which necessitates native support for out-of-order placement within the NIC.

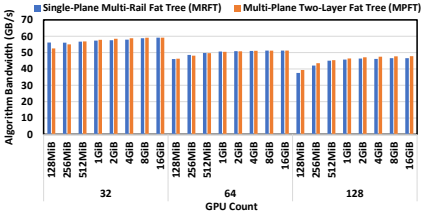


Figure 5: NCCL all-to-all performance from 32 to 128 GPUs for MRFT and MPFT networks.

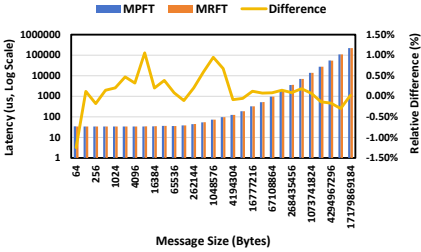


Figure 6: Latency comparison between MPFT and MRFT networks in NCCL all-to-all test under different message sizes, showing that their performance is nearly identical.

MPFT and MRFT in Table 4. MFU (Model Flops Utilization) is calculated based on BF16 peak performance. Causal MFU only takes into account the flops of the lower triangle of the attention matrix (in line with FlashAttention[19, 20]), while non-causal MFU includes the flops of the whole attention matrix (in line with Megatron [47]). 1F, 1B, and 1W denote forward time, input backward time, and weight backward time, respectively. When training the V3

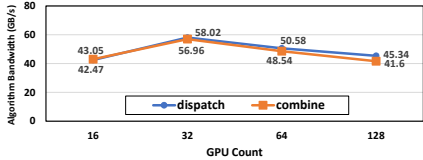


Figure 7: DeepEP performance on MPFT: The EP dispatch and combine kernel communicates across 16 to 128 GPUs using all-to-all. Each GPU processes 4096 tokens. The observed throughput nearly saturates the 400Gps NIC bandwidth.

model on 2048 GPUs, the performance of MPFT is nearly identical to that of MRFT, with observed differences falling within normal fluctuations and measurement error.

5.2 Low Latency Networks

In our model inference, large-scale EP relies heavily on all-to-all communication, which is highly sensitive to both bandwidth and latency. Consider a typical scenario discussed in Section 2.3.2, with a network bandwidth of 50GB/s, the data transfer should ideally take approximately 120 μ s. Therefore, the intrinsic network latencies on the order of microseconds can critically impact system performance, making their effects non-negligible.

5.2.1 IB or RoCE. As shown in Table 5, IB consistently achieves lower latency, making it the preferred choice for latency-sensitive workloads such as distributed training and inference. Although IB has superior latency performance compared to RDMA over Converged Ethernet (RoCE), it comes with certain limitations:

5.2.2 Recommendations for RoCE Improvements. While RoCE has the potential to be a cost-effective alternative to IB, its current limitations in latency and scalability prevent it from fully meeting the demands of large-scale AI systems. Below, we outline specific recommendations for improving RoCE:

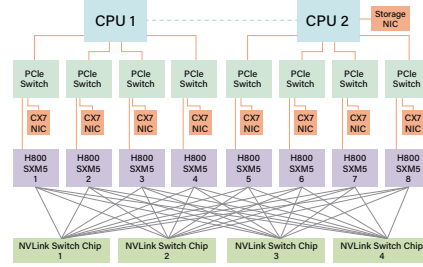


Figure 2: H800 节点互连。

4.3 Model Co-Design:

Node-Limited Routing

H800 架构中，纵向扩展（节点内）与横向扩展（节点间）通信之间的带宽差异约为 4:1。具体而言，NVLink 提供 200GB/s 带宽（其中实际可实现约 160GB/s），而每个 400Gbps IB 网卡仅提供 50GB/s 带宽（考虑到小消息大小和延迟影响，我们使用 40GB/s 作为有效带宽）。为了平衡并充分利用更高的节点内带宽，模型架构与硬件协同设计，特别是在 **TopK Expert Selection Strategy** 中。

考虑一个包含 8 个节点（总共 64 个 GPU）和 256 个路由专家（每个 GPU 4 个专家）的设置。对于 DeepSeek-V3，每个令牌被路由到一个共享专家和 8 个路由专家。如果其 8 个目标专家分布在所有 8 个节点上，则通过 IB 的通信时间将为 $8t$ ，其中 t 表示通过 IB 发送一个令牌的时间。然而，通过利用更高的 NVLink 带宽，路由到同一节点的令牌可以通过 IB 发送一次，然后通过 NVLink 转发给节点内的其他 GPU。NVLink 转发实现了 IB 流量的去重。当给定令牌的目标专家分布在 M 个节点上时，去重后的 IB 通信成本将减少到 Mt ($M < 8$)。

由于 IB 流量仅取决于 M ，DeepSeek-V3 为 TopK 专家选择策略引入了 **Node-Limited Routing**。具体来说，我们将 256 个路由专家分为 8 组，每组 32 个专家，并将每个组部署在单个节点上。在此部署基础上，我们通过算法确保每个令牌最多被路由到 4 个节点。这

种方法缓解了 IB 通信的瓶颈，并提升了训练期间的有效通信带宽。

4.4 Scale-Up and Scale-Out Convergence

4.4.1 Limitations of Current Implementations. 虽然节点受限路由策略降低了通信带宽需求，但由于节点内（NVLink）与节点间（IB）互连带宽的差异，它使得通信流水线内核的实现变得复杂。为了最大化在线推理的吞吐量，我们完全通过 NIC RDMA 执行 EP 全对全通信，避免了 SM 资源争用并提高了计算效率。这凸显了 RDMA 异步通信模型在重叠计算与通信方面的优势。以下是 SM 在 EP 通信期间当前执行的关键任务，特别是针对 combine 阶段的 reduce 操作和数据类型转换。将这些任务卸载到专用的通信硬件上，可以释放 SM 以执行计算内核，从而显著提升整体效率：

- **Forwarding Data:** 聚合同一节点内、目标为多个 GPU、在 IB 与 NVLink 域之间的 IB 流量。
- **Data Transport:** 在 RDMA 缓冲区（已注册的 GPU 内存区域）与输入/输出缓冲区之间移动数据。
- **Reduce Operations:** 执行 EP all-to-all combine 通信所需的 reduce 操作。
- **Managing Memory Layouts:** 处理跨 IB 和 NVLink 域的、用于分块数据传输的细粒度内存布局。
- **Data Type Cast:** 在 all-to-all 通信前后进行数据类型转换。
- **Unified Network Adapter:** 设计连接到统一纵向扩展与横向扩展网络的网络接口卡或 I/O 芯片。这些适配器还应支持基本的交换机功能，例如将数据包从横向扩展网络转发到纵向扩展网络内的特定 GPU。这可以通过使用单个本地标识符或具有基于策略路由的 IP 地址来实现。
- **Dedicated Communication Co-Processor:** 引入专用的协处理器或可编程组件（例如 I/O 芯片）来处理网络流量。该组件应卸载 GPU 流式多处理器的数据包处理任务，为高效的缓冲区管理提供硬件加速的内存拷贝，并且至关重要，以类似于张量内

Table 4: Training metric comparison between MPFT and MRFT networks.

Metric	MPFT	MRFT
tokens/day (B)	272.80	272.52
time/step (s)	19.926	19.946
1F (s)	1.13	1.13
bubble (s)	2.06	2.03
1B (s)	1.99	1.99
1W (s)	0.48	0.48
1F1B (s)	13.95	14.00
opt (s)	0.29	0.31
TFLOPS (non-causal)	432	432
TFLOPS (causal)	385	385
MFU (non-causal)	43.73%	43.68%
MFU (causal)	38.94%	38.90%

Table 5: CPU side end-to-end latency comparison between IB, RoCE, and intra-node NVLink for 64B data transmission.

Link Layer	Same Leaf	Cross Leaf
RoCE	3.6us	5.6us
InfiniBand	2.8us	3.7us
NVLink	3.33us	-

- (1) **Specialized Low-Latency RoCE Switches:** We recommend that Ethernet vendors develop RoCE switches specifically optimized for RDMA workloads by removing unnecessary Ethernet features. The Slingshot architecture [22] exemplifies how Ethernet-based designs can achieve latency performance comparable to IB. Similarly, recent innovations from Broadcom [13], including the AI Forwarding Header (AIFH) and upcoming low-latency Ethernet switches, demonstrate the feasibility of high-performance Ethernet fabrics tailored for AI. We are looking forward to continuing innovation in this direction.
- (2) **Optimized Route Policy:** As shown in Figure 8, the default Equal-Cost Multi-Path (ECMP) routing policy in RoCE struggles to distribute traffic efficiently across interconnects, leading to severe congestion performance degradation in NCCL collective communication tests. LLM training traffic, such as in DP (Data Parallelism), tends to lack randomness, causing multiple flows to converge on the same interconnect link. In contrast, Adaptive Routing (AR) [34] can significantly enhance network performance by dynamically spraying packets across multiple paths. While static routing—based on manually configured route tables—can avoid link conflicts for specific destinations, it lacks flexibility. For large-scale all-to-all communication, adaptive routing offers superior performance and scalability.
- (3) **Improved Traffic Isolation or Congestion Control Mechanisms:** Current RoCE switches support only a limited number of priority queues, which are insufficient for complex AI workloads involving concurrent communication patterns such as EP’s all-to-all and DP’s all-reduce. In such mixed workloads, all-to-all traffic can cause incast congestion due to bursty many-to-one transfers, potentially degrading overall network performance. To address incast’s influence on other traffic, one approach is to adopt virtual output queuing (VOQ), assigning a dedicated virtual queue to each QP to isolate traffic flows. Alternatively, more effective congestion control (CC) mechanisms such as

RTT-based CC (RTTCC) or user-programmable CC (PCC) can be employed, enabling NIC-switch co-optimization to maintain low latency and high throughput under dynamic traffic conditions.

- (4) **Interconnect Failures:** High-performance interconnects (e.g., IB and NVLink) are prone to intermittent disconnections, which can disrupt node-to-node communication. This is especially harmful in communication-heavy workloads like EP, where even brief interruptions may lead to significant performance drops or job failures.
- (5) **Single Hardware Failures:** Node crashes, GPU failures, or ECC (Error-Correcting Code) memory errors can compromise long-running training jobs, often requiring costly restarts. The impact of such failures escalates in large-scale deployments, where the probability of a single-point failure increases proportionally with system size.
- (6) **Silent Data Corruption:** Errors undetected by ECC mechanisms, such as multi-bit memory flips or computational inaccuracies, pose a significant risk to model quality. These errors are particularly insidious in long-running tasks, as they can propagate undetected and corrupt downstream computations. Current mitigation strategies rely on application-level heuristics, which are insufficient for ensuring system-wide robustness.
- (7) **Co-Packaged Optics:** Incorporating silicon photonics enables scalable higher bandwidth scalability and enhanced energy efficiency, both are critical for large-scale distributed systems.
- (8) **Lossless Network:** Credit-Based Flow Control (CBFC) mechanisms ensures lossless data transmission, yet naively triggering flow control can induce severe head-of-line blocking. Therefore, it is imperative to deploy advanced, endpoint-driven congestion control (CC) algorithms that proactively regulate injection rates and avert pathological congestion scenarios.
- (9) **Adaptive Routing:** As underscored in Section 5.2.2, future network should standardize the adoption of dynamic routing schemes—such as packet spraying and congestion-aware path selection—that continuously monitor real-time network conditions and intelligently redistribute traffic. These adaptive strategies are particularly effective in alleviating hotspots and mitigating bottlenecks during collective communication workloads, including all-to-all and reduce-scatter operations.
- (10) **Efficient Fault-Tolerant Protocols:** Robustness against failures can be significantly enhanced through the deployment of self-healing protocols, redundant ports, and rapid failover techniques. For instance, link-layer retry mechanisms and selective retransmission protocols prove indispensable in scaling reliability across large networks, minimizing downtime and ensuring seamless operation despite intermittent failures.
- (11) **Dynamic Resource Management:** To handle mixed workloads effectively, future hardware should enable dynamic bandwidth allocation and traffic prioritization. For example, inference tasks should be isolated from training traffic in unified clusters, ensuring responsiveness for latency-sensitive applications.
- (12) **DRAM-Stacked Accelerators:** Leveraging advanced 3D stacking technologies, DRAM dies can be vertically integrated atop a logic die, thereby enabling exceptionally high memory bandwidth, ultra-low latency, and a practical memory capacity (though stack-limited). This architectural paradigm proves remarkably

- 存加速器的方式加速内存加载/存储操作，从而以最小的资源消耗饱和带宽。
- **Flexible Forwarding, Broadcast and Reduce Mechanisms:** 硬件应支持跨纵向扩展和横向扩展网络的灵活转发、广播操作（用于 EP 分发）和归约操作（用于 EP 合并）——这反映了我们当前基于 GPU 流式多处理器的实现方式。这不仅可以提高有效带宽，还会降低网络特定操作的计算复杂度。
- **Hardware Synchronization Primitives:** 提供细粒度的硬件同步指令，以在硬件层面处理内存一致性问题或数据包乱序到达问题。这将消除对基于软件的同步机制（如 RDMA 完成事件）的需求，这些机制会引入额外的延迟并增加编程复杂性。具有获取/释放机制的内存语义通信是一种有前景的实现方式。
- **Dynamic NVLink/PCIe Traffic Prioritization:** 硬件应支持根据流量类型进行动态优先级划分。例如，与 EP、TP 和 KV 缓存传输相关的流量应分配不同的优先级，以最大化互连效率。对于 PCIe，向用户级编程暴露流量类别就足够了。
- **I/O Die Chiplet Integration:** 将网络接口卡直接集成到 I/O 芯片中，并将其连接到同一封装内的计算芯片，而不是通过传统的 PCIe 连接，这将显著降低通信延迟并缓解 PCIe 带宽争用。
- **CPU-GPU Interconnects within the Scale-Up Domain:** 为了进一步优化节点内通信，CPU 和 GPU 应使用 NVLink 或类似的专用高带宽结构互连，而不是仅仅依赖 PCIe。与将网络接口卡集成到 I/O 芯片所带来的好处类似，这种方法可以显著改善训练和推理期间在 GPU 与 CPU 内存之间卸载参数或 KV 缓存等场景。
- **Subset of Multi-Rail Fat-Tree (MRFT):** MPFT 拓扑构成了更广泛的 MRFT 架构的一个特定子集。因此，NVIDIA 和 NCCL 为多轨网络开发的现有优化可以在多平面网络部署中无缝利用。此外，NCCL 对 PXN [54] 技术的支持解决了平面间固有的隔离挑战，即使在平面之间缺乏直接互连的情况下也能实现高效通信。

- **Cost Efficiency:** 如表 3 所示，多平面网络使用两层胖树拓扑可支持超过 1 万个端点，与三层胖树相比显著降低了网络成本。每个端点的成本甚至比高性价比比的 Slim Fly 拓扑 [12] 更具竞争力。
- **Traffic Isolation:** 每个平面独立运行，确保一个平面的拥塞不会影响其他平面。这种隔离提高了整体网络稳定性，并防止级联性能下降。
- **Latency Reduction:** 如我们的实验所示，两层拓扑比三层胖树实现了更低的延迟。这使其特别适用于对延迟敏感的应用，例如基于 MoE 的训练和推理。
- **Robustness:** 如图 4 所示，多端口网络接口卡提供多个上行链路，因此单端口故障不会中断连接，并且可以实现快速、透明的故障恢复。
- **Cost:** IB 硬件比 RoCE 解决方案昂贵得多，这限制了其广泛采用。
- **Scalability:** IB 交换机通常每台只支持 64 个端口，而 RoCE 交换机通常支持 128 个端口。这限制了基于 IB 的集群的可扩展性，特别是在大规模部署中。
- **Specialized Low-Latency RoCE Switches:**
- **Optimized Route Policy:** 如图??所示，RoCE 中默认的等价多路径 (ECMP) 路由策略难以在互连链路上高效分配流量，导致 NCCL 集合通信测试中出现严重的拥塞性能下降。LLM 训练流量，例如在数据并行 (DP) 中，往往缺乏随机性，导致多个流汇聚到同一条互连链路上。相比之下，自适应路由 (AR) [34] 可以通过动态地将数据包分散到多条路径上，显著提升网络性能。虽然基于手动配置路由表的静态路由可以避免特定目的地的链路冲突，但缺乏灵活性。对于大规模的全对全通信，自适应路由提供了更优的性能和可扩展性。
- **Improved Traffic Isolation or Congestion Control Mechanisms:**
- **Interconnect Failures:** 高性能互连（例如 IB 和 NVLink）容易出现间歇性断开连接，这会中断节点间通信。这在通信密集型工作负载（如专家并行）中尤其有害，即使短暂的中断也可能导致显著的性能下降或作业失败。

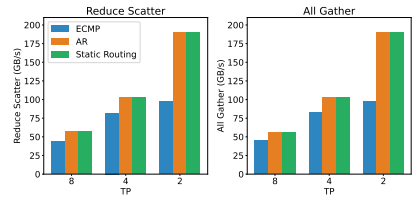


Figure 8: RoCE network bandwidth of AllGather and ReduceScatter communication primitives under different routing methods (ECMP, AR, Static Routing) and TP dimensions.

advantageous for ultra-fast inference in MoE models, where memory throughput is a critical bottleneck. Architectures such as SeDRAM[73] exemplify the potential of this approach, delivering unprecedented performance for memory-bound workloads.

(13) **System-on-Wafer (SoW):** Wafer-scale integration [50] can maximize computational density and memory bandwidth, addressing the needs of ultra-large-scale models.

5.2.3 InfiniBand GPU Direct Async (IBGDA). We utilize IBGDA [2, 57] to reduce latency in network communications. Traditionally, network communication involves the creation of a CPU proxy thread: once the GPU has prepared the data, it must notify the CPU proxy, which then populates the control information for the work request (WR) and signals the NIC via a doorbell mechanism to initiate data transmission. This process introduces additional communication overhead.

IBGDA addresses this issue by allowing the GPU to directly fill the WR content and write to the RDMA doorbell MMIO address. By managing the entire control plane within the GPU, IBGDA eliminates the significant latency overhead associated with GPU-CPU communication. Moreover, when sending a large number of small packets, the control plane processor can easily become a bottleneck. Since GPUs have multiple parallel threads, the sender can leverage these threads to distribute the workload, thereby avoiding such bottlenecks. A range of works—including our DeepEP [79]—have leveraged IBGDA and reported substantial performance gains [1, 15, 80]. We therefore advocate for such capabilities to be widely supported across accelerator devices.

6 Discussion and Insights for Future Hardware Architecture Design

Building on the previous sections, we summarize key architectural insights and outline future directions for hardware design tailored to large-scale AI workloads.

Section 2.3.2 highlighted the importance of large-scale scale-up networks for accelerating model inference. Section 3 discussed the necessity of efficient support for low-precision computation and communication. Section 4 explored the convergence of scale-up and scale-out architectures, along with several proposed enhancements. Section 5 focused on multi-plane network topologies and identified key improvements needed for Ethernet-based interconnects.

Together, these sections identify hardware limitations in concrete application contexts and offer corresponding suggestions. Building on that foundation, this section expands the discussion to broader considerations and proposes forward-looking directions for future hardware architecture design.

6.1 Robustness Challenges

6.1.1 Limitations:

6.1.2 Suggestions for Advanced Error Detection and Correction. To mitigate risks associated with silent corruption, hardware must incorporate advanced error detection mechanisms beyond traditional ECC. Techniques such as checksum-based validation or hardware-accelerated redundancy checks can provide higher reliability for large-scale deployments.

Furthermore, hardware vendors should deliver comprehensive diagnostic toolkits to end users, empowering them to rigorously verify the integrity of their systems and proactively identify any latent silent data corruption. Such toolkits, when embedded as part of the standard hardware package, foster transparency and enable continuous validation throughout the operational lifecycle, thereby bolstering overall system trustworthiness.

6.2 CPU Bottlenecks and Interconnects

While accelerator design often takes center stage, CPUs remain essential for coordinating computation, managing I/O, and sustaining system throughput. However, current architectures face several critical bottlenecks:

First, as discussed in Section 4.5, the PCIe interface between CPUs and GPUs often becomes a bandwidth bottleneck, particularly during large-scale parameter, gradient, or KV cache transfers. To mitigate this, future systems should adopt direct CPU-GPU interconnects—such as NVLink or Infinity Fabric—or integrate both CPUs and GPUs into the scale-up domain, thereby eliminating intra-node bottlenecks.

In addition to PCIe limitations, sustaining such high data transfer rates also requires exceptionally high memory bandwidth. For example, saturating 160 lanes of PCIe 5.0 demands over 640 GB/s per node, translating to a memory bandwidth requirement of approximately 1 TB/s per node—posing a significant challenge for conventional DRAM architectures.

Lastly, latency-sensitive tasks such as kernel launches and network processing demand high single-core CPU performance, typically requiring base frequencies above 4 GHz. Furthermore, modern AI workloads require sufficient CPU cores per GPU to prevent control-side bottlenecks. For chiplet-based architectures, additional cores are needed to support cache-aware workload partitioning and isolation.

6.3 Toward Intelligent Networks for AI

To meet the demands of latency-sensitive workloads, future interconnects must prioritize both low latency and intelligent networks:

- **Single Hardware Failures:** 节点崩溃、GPU 故障或纠错码 (ECC) 内存错误可能会损害长时间运行的训练作业，通常需要代价高昂的重启。此类故障的影响在大规模部署中会升级，因为单点故障的概率随系统规模成比例增加。
- **Silent Data Corruption:** ECC 机制未检测到的错误，例如多位内存翻转或计算不准确，对模型质量构成重大风险。这些错误在长时间运行的任务中尤其隐蔽，因为它们可能未被发现地传播并破坏下游计算。当前的缓解策略依赖于应用层面的启发式方法，这不足以确保系统范围的鲁棒性。
- **Co-Packaged Optics:** 集成硅光子学可以实现可扩展的更高带宽和增强的能源效率，这两者对于大规模分布式系统都至关重要。
- **Lossless Network:** 基于信用的流控 (CBFC) 机制确保了无损数据传输，但简单地触发流控可能导致严重的队头阻塞。因此，必须部署先进的、端点驱动的拥塞控制 (CC) 算法，主动调节注入速率并避免病态拥塞场景。
- **Adaptive Routing:** 如第 5.2.2 节所强调的，未来的网络应标准化采用动态路由方案——例如数据包喷洒和拥塞感知路径选择——这些方案持续监控实时网络状况并智能地重新分配流量。这些自适应策略在缓解集合通信工作负载（包括全对全和规约分散操作）期间的热点和瓶颈方面特别有效。
- **Efficient Fault-Tolerant Protocols:** 通过部署自愈协议、冗余端口和快速故障转移技术，可以显著增强对故障的鲁棒性。例如，链路层重试机制和选择性重传协议被证明对于扩展大型网络的可靠性、最小化停机时间以及确保在间歇性故障下的无缝运行是不可或缺的。
- **Dynamic Resource Management:** 为了有效处理混合工作负载，未来的硬件应支持动态带宽分配和流量优先级划分。例如，在统一集群中，应将推理任务与训练流量隔离，以确保对延迟敏感应用的响应性。
- **DRAM-Stacked Accelerators:** 利用先进的 3D 堆叠技术，DRAM 芯片可以垂直集成在逻辑芯片之上，从

而实现极高的内存带宽、超低延迟和实用的内存容量（尽管受堆叠限制）。这种架构范式对于 MoE 模型中的超快速推理证明非常有利，其中内存吞吐量是一个关键瓶颈。诸如 SeDRAM[73] 之类的架构展示了这种方法的潜力，为内存受限的工作负载提供了前所未有的性能。

- **System-on-Wafer (SoW):** 晶圆级集成 [50] 可以最大化计算密度和内存带宽，满足超大规模模型的需求。

4.4.2 Suggestions: 为了解决这些低效问题，我们强烈建议未来的硬件应将节点内（纵向扩展）和节点间（横向扩展）通信集成到一个统一的框架中。通过集成用于网络流量管理以及 NVLink 和 IB 域之间无缝转发的专用协处理器，此类设计可以降低软件复杂性并最大化带宽利用率。例如，DeepSeek-V3 中采用的节点受限路由策略可以通过硬件对动态流量去重的支持得到进一步优化。

我们也认识到新兴的互连协议，例如超以太网联盟 (UEC) [17, 18]、超加速器链路 (UALink) [16]，两者都有望推动纵向扩展和横向扩展通信的进步。最近，统一总线 (UB) [49] 引入了一种实现纵向扩展和横向扩展融合的新方法。第 ?? 节进一步探讨了 UEC 和 UALink 提出的几项技术创新。然而，在本节中，我们的主要关注点是在编程框架层面实现纵向扩展和横向扩展的融合：

通过实施这些建议，未来的硬件设计可以显著提高大规模分布式人工智能系统的效率，同时简化软件开发。

4.5 Bandwidth Contention and Latency

4.5.1 Limitations: 此外，当前的硬件缺乏在 NVLink 和 PCIe 上为不同类型流量动态分配带宽的灵活性。例如，在推理过程中，将 KV 缓存数据从 CPU 内存传输到 GPU 可能会消耗数十 GB/s 的带宽，使 PCIe 带宽饱和。如果 GPU 同时使用 IB 进行 EP 通信，KV 缓存传输与 EP 通信之间的这种争用会降低整体性能并导致延迟峰值。

6.4 Discussion on Memory-Semantic Communication and Ordering Issue

Inter-node communication using load/store memory semantics is efficient and programmer-friendly, but current implementations are hampered by memory ordering challenges. For example, after writing data, the sender must issue an explicit memory barrier (fence) before updating a flag to notify the receiver, ensuring data consistency. This strict ordering introduces additional round-trip time (RTT) latency and can stall the issuing thread, impeding in-flight stores and reducing throughput. Similar out-of-order synchronization issues arise in message-semantic RDMA; for instance, performing RDMA atomic add operations with packet spraying after regular RDMA writes on InfiniBand or NVIDIA BlueField-3 can incur additional RTT latency.

To address these, we advocate for hardware support that offers built-in ordering guarantees for memory-semantic communication. Such consistency should be enforced both at the programming level (e.g., via acquire/release semantics) and by hardware at the receiver, enabling in-order delivery without added overhead.

Several approaches are possible. For example, the receiver can buffer atomic messages and use packet sequence numbers (PSN) to ensure in-order processing; this method is straightforward and effective for maintaining correctness. Alternatively, a region-based acquire/release (RAR) mechanism is particularly attractive, as it enables true acquire/release semantics for remote memory access and offers greater flexibility for a range of workloads. In this approach, the receiver hardware maintains lightweight metadata—such as bitmaps or region-based counters—to track the state of memory regions. Acquire and release operations are scoped to specific address ranges, enabling efficient, hardware-enforced ordering without explicit sender-side fences. Notably, both approaches are amenable to implementation on the NIC or I/O die and are applicable to both memory-semantic and message-semantic RDMA primitives, thereby broadening their practical utility.

6.5 In-Network Computation and Compression

EP involves two critical all-to-all stages—**dispatch** and **combine**—that present significant opportunities for in-network optimization. The **dispatch** stage resembles a small-scale multicast operation, where a single message must be forwarded to multiple target devices. A hardware-level protocol enabling automatic packet replication and forwarding to multiple destinations could drastically reduce communication overhead and improve efficiency.

The **combine** stage, acting as a small-scale reduction operation, could benefit from in-network aggregation techniques. However, due to the small reduction scope and imbalanced workload in EP combine, implementing in-network aggregation in a flexible manner is challenging.

Moreover, as highlighted in Section 3.2, LogFMT enables low-precision token transmission with minimal impact on model performance. Incorporating LogFMT natively within network hardware could further optimize communication by increasing entropy density and reducing bandwidth usage. Hardware-accelerated compression and decompression would allow seamless integration of LogFMT into distributed systems, enhancing overall throughput.

6.6 Memory-Centric Innovations

6.6.1 Limitations of Memory Bandwidth. The exponential growth in model sizes has outpaced advancements in high-bandwidth memory (HBM) technology. This disparity creates a memory bottleneck, particularly in attention-heavy architectures like Transformers.

6.6.2 Suggestions:

7 Conclusion

DeepSeek-V3 exemplifies the transformative potential of hardware-software co-design in advancing the scalability, efficiency, and robustness of large-scale AI systems. By addressing the limitations of current hardware architectures and proposing actionable recommendations, this paper provides a roadmap for the next generation of AI-optimized hardware. These innovations will be critical as AI workloads continue to grow in complexity and scale, driving the future of intelligent systems.

References

- [1] Elena Agostini, Davide Rossetti, and Sreeram Potluri. 2017. Offloading Communication Control Logic in GPU Accelerated Applications. In *2017 17th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGRID)*. 248–257. <https://doi.org/10.1109/CCGRID.2017.29>
- [2] E. Agostini, D. Rossetti, and S. Potluri. 2018. GPUDirect Async: Exploring GPU synchronous communication techniques for InfiniBand clusters. *J. Parallel and Distrib. Comput.* 114 (2018), 28–45. <https://doi.org/10.1016/j.jpdc.2017.12.007>
- [3] AI@Meta. 2024. Llama 3 Model Card. https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md
- [4] AI@Meta. 2024. Llama 3.1 Model Card. https://github.com/meta-llama/llama-models/blob/main/models/llama3_1/MODEL_CARD.md
- [5] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. 2023. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *arXiv preprint arXiv:2305.13245* (2023).
- [6] AMD. 2025. AMD Ryzen AI Max+ PRO 395: Designed to power a new generation of compact Copilot+ PC workstations. <https://www.amd.com/en/products/processors/laptop/ryzen-pro-ai-max-pro-390-series/amd-ryzen-ai-max-plus-pro-395.html>
- [7] Wei An, Xiao Bi, Guanting Chen, Shanhua Chen, Chengqi Deng, Honghui Ding, Kai Dong, Qushi Du, Wenjun Gao, Kang Guan, Jianzhong Guo, Yongqiang Guo, Zhe Fu, Ying He, Panpan Huang, Jiashi Li, Wenfeng Liang, Xiaodong Liu, Xin Liu, Yiyuan Liu, Yuxuan Liu, Shanghao Lu, Xuan Lu, Xiaotao Nie, Tian Pei, Junjie Qu, Hui Qu, Zehui Ren, Zhangli Shi, Xuecheng Su, Xiaowen Sun, Yixuan Tan, Minghui Tang, Shiyu Wang, Yaohui Wang, Yongji Wang, Ziwei Xie, Yiliang Xiong, Yanhong Xu, Shengfeng Ye, Shiping Yu, Yukun Zha, Liyue Zhang, Haowei Zhang, Mingchuan Zhang, Wentao Zhang, Yichao Zhang, Chenggang Zhao, Yao Zhao, Shangyan Zhou, Shunfeng Zhou, and Yuheng Zou. 2024. Fire-Flyer AI-HPC: A Cost-Effective Software-Hardware Co-Design for Deep Learning. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–23. <https://doi.org/10.1109/SC41406.2024.00089>
- [8] Anthropic. 2024. Claude 3.5 Sonnet. <https://www.anthropic.com/news/claude-3-5-sonnet>
- [9] Anthropic. 2025. Claude 3.7 Sonnet and Claude Code. <https://www.anthropic.com/news/claude-3-7-sonnet>
- [10] Apple. 2024. Apple introduces M4 Pro and M4 Max. <https://www.apple.com/newsroom/2024/10/apple-introduces-m4-pro-and-m4-max/>
- [11] Iz Beltagy, Matthew E. Peters, and Arman Cohan. 2020. Longformer: The Long-Document Transformer. *arXiv:2004.05150* (2020).
- [12] Nils Blach, Maciej Besta, Daniele De Sensi, Jens Domke, Hussein Harake, Shigang Li, Patrick Iff, Marek Konieczny, Kartik Likhitha, Ales Kubicek, Marcel Ferrari, Fabrizio Petrini, and Torsten Hoefler. 2025. A high-performance design, implementation, deployment, and evaluation of the slim fly network. In *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation* (Santa Clara, CA, USA) (NSDI/24). USENIX Association, USA, Article 57, 20 pages.
- [13] Broadcom. 2025. Scale Up Ethernet Framework. <https://docs.broadcom.com/doc/scale-up-ethernet-framework>
- [14] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21–27, 2024*. OpenReview.net. <https://openreview.net/forum?id=PEpbUohfJy>

5 Large Scale Network Driven Design

5.1 Network Co-Design: Multi-Plane Fat-Tree

在 DeepSeek-V3 的训练过程中,我们部署了一个 **Multi-Plane Fat-Tree (MPFT)** 规模的横向扩展网络,如图 3 所示。每个节点配备八个 GPU 和八个 IB 网卡,每个 GPU-网卡对分配到一个独立的网络平面。此外,每个节点还有一个 400 Gbps 以太网 RoCE 网卡,连接到独立的存储网络平面,用于访问 3FS [30] 分布式文件系统。在横向扩展网络中,我们使用了 64 端口 400G IB 交换机,使得该拓扑理论上可支持多达 16,384 个 GPU,同时保留了双层网络在成本和延迟方面的优势。然而,由于政策和监管限制,最终仅部署了略多于两千个 GPU。

此外,由于当前 IB ConnectX-7 的限制,我们部署的 MPFT 网络并未完全实现预想的架构。理想情况下,如图 4 所示,每个网卡应具有多个物理端口,每个端口连接到一个独立的网络平面,但通过端口绑定技术,这些端口在用户面前作为一个单一的逻辑接口呈现。从用户的角度看,单个队列对 (QP) 可以无缝地通过所有可用端口发送和接收消息,类似于数据包喷洒。因此,源自同一 QP 的数据包可能经由不同的网

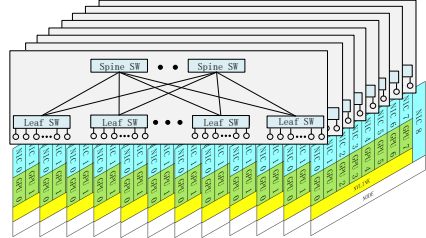


Figure 3: 八平面双层胖树横向扩展网络: 每个 GPU 和 IB 网卡对属于一个网络平面。跨平面流量必须使用另一个网卡以及 PCIe 或 NVLink 进行节点内转发。

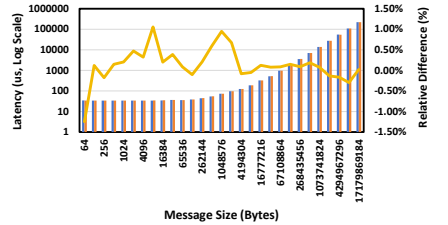
Table 3: 网络拓扑比较。成本估算源自 Slim Fly (SF) 论文 [12] 中的方法。DF 表示经典的蜻蜓拓扑 [22, 46, 65]。

Metric	FT2	MPFT	FT3	SF	DF
Endpoints	2,048	16,384	65,536	32,928	261,632
Switches	96	768	5,120	1,568	16,352
Links	2,048	16,384	131,072	32,928	384,272
Cost [M\$]	9	72	491	146	1,522
Cost/Endpoint [k\$]	4.39	4.39	7.5	4.4	5.8

路路径传输,并以乱序到达接收端,这便需要在网卡内部原生支持乱序放置功能,以保证消息一致性并维持正确的顺序语义。例如, InfiniBand ConnectX-8 原生支持四个平面。未来的网卡若能完全支持先进的多平面能力,将非常有利于双层胖树网络有效地扩展到更大的 AI 集群。总体而言,多平面架构在故障隔离、鲁棒性、负载均衡以及大规模系统可扩展性方面具有显著优势。

5.1.1 Advantages of Multi-Plane Fat-Tree Network.

需要指出的是,由于当前 400G NDR InfiniBand 的限制,跨平面通信需要进行节点内转发,这会在推理过程中引入额外的延迟。如果未来的硬件能够实现如前所述的纵向扩展与横向扩展网络融合,这一延迟将显著降低,从而进一步提升多平面网络的可行性。



5.1.2 Performance Analysis. 为了验证多平面网络设计的有效性,我们在集群上进行了真实环境实验,通过修改集群的网络拓扑来比较 **Multi-Plane Two-Layer Fat Tree (MPFT)** 和 **Single-Plane Multi-Rail Fat Tree (MRFT)** 的性能。以下是我们实验中的关键发现:

Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Huazuo Gao, Jiashi Li, Liyue Zhang, Panpan Huang, Shangyan Zhou, Shiroong Ma, Wenfeng Liang, Ying He, Yuqing ISCA '25, June 21–25, 2025, Tokyo, Japan

[15] Shaoyuan Chen, Wencong Xiao, Yutong Lin, Mingxing Zhang, Yingdi Shan, Jinlei Jiang, Kang Chen, and Yongwei Wu. 2025. Efficient Heterogeneous Large Language Model Decoding with Model-Attention Disaggregation. *arXiv:2405.01814* [cs.LG]. <https://arxiv.org/abs/2405.01814>

[16] ULTRA ACCELERATOR LINK CONSORTIUM. 2025. Introducing UALink 200G 1.0 Specification. https://ulinkconsortium.org/wp-content/uploads/2025/04/UALink-1.0-White-Paper_FINAL.pdf

[17] Ultra Ethernet Consortium. 2023. Overview of and Motivation for the Forthcoming Ultra Ethernet Consortium Specification. <https://ultraethernet.org/wp-content/uploads/sites/20/2023/10/23.07.12-UEC-1.0-Overview-FINAL-WITH-LOGO.pdf>

[18] Ultra Ethernet Consortium. 2024. UEC Progresses Towards v1.0 Set of Specifications. <https://ultraethernet.org/uec-progresses-towards-v1-0-set-of-specifications/>

[19] Tri Dao. 2023. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning.

[20] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems*.

[21] Tri Dao and Albert Gu. 2024. Transformers are SSMs: generalized models and efficient algorithms through structured state space duality. In *Proceedings of the 41st International Conference on Machine Learning (Vienna, Austria) (ICML '24)*. JMLR.org, Article 399, 31 pages.

[22] Daniele De Sensi, Salvatore Di Girolamo, Kim H. McMahon, Duncan Rothwell, and Torsten Hoefler. 2020. An In-Depth Analysis of the Slingshot Interconnect. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–14. <https://doi.org/10.1109/SC41405.2020.00039>

[23] DeepSeek-AI. 2024. DeepSeek-Coder-V2: Breaking the Barrier of Closed-Source Models in Code Intelligence. *CoRR* abs/2406.11931 (2024). <https://doi.org/10.48550/arXiv.2406.11931>

[24] DeepSeek-AI. 2024. DeepSeek LLM: Scaling Open-Source Language Models with Longtermism. *CoRR* abs/2401.02954 (2024). <https://doi.org/10.48550/arXiv.2401.02954>

[25] DeepSeek-AI. 2024. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. *CoRR* abs/2405.04434 (2024). <https://doi.org/10.48550/arXiv.2405.04434>

[26] DeepSeek-AI. 2024. DeepSeek-V3 Technical Report. (2024). <https://arxiv.org/abs/2412.19437> [cs.LG]. <https://arxiv.org/abs/2412.19437>

[27] DeepSeek-AI. 2024. DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models. *CoRR* abs/2401.06066 (2024). <https://doi.org/10.48550/arXiv.2401.06066>

[28] DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv:2501.12948* [cs.CL]. <https://arxiv.org/abs/2501.12948>

[29] DeepSeek-AI. 2025. DualPipe: A bidirectional pipeline parallelism algorithm for computation-communication overlap in V3/R1 training. <https://github.com/deepseek-ai/dualpipe>

[30] DeepSeek-AI. 2025. Fire-Flyer File System. <https://github.com/deepseek-ai/3FS>

[31] DeepSeek-AI. 2025. Profiling Data in DeepSeek Inference. <https://github.com/deepseek-ai/profile-data?tab=readme-ov-file#inference>

[32] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2022. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323* (2022).

[33] Adithya Gangidi, Rui Miao, Shengbao Zheng, Sai Jayesh Bondu, Guilherme Goes, Hany Morsy, Rohit Puri, Mohammad Rifathi, Ashmitha Jeevaraj Shetty, Jingyi Yang, Shuang Zhang, Mikel Jimenez Fernandez, Shashidhar Gandham, and Hongyi Zeng. 2024. RDMA over Ethernet for Distributed Training at Meta Scale. In *Proceedings of the ACM SIGCOMM 2024 Conference (Sydney, NSW, Australia) (ACM SIGCOMM '24)*. Association for Computing Machinery, New York, NY, USA, 57–70. <https://doi.org/10.1145/3651890.3672233>

[34] Patrick Georffroy and Torsten Hoefler. 2008. Adaptive Routing Strategies for Modern High Performance Networks. In *2008 16th IEEE Symposium on High Performance Interconnects*. 165–172. <https://doi.org/10.1109/HOTI.2008.21>

[35] Amir Gholami, Zhewei Yao, Sehoon Kim, Coleman Hooper, Michael W. Mahoney, and Kurt Keutzer. 2024. AI and Memory Wall. *IEEE Micro* 44, 03 (May 2024), 33–39. <https://doi.org/10.1109/MM.2024.337363>

[36] Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. 2024. Better & Faster Large Language Models with Multi-token Prediction. In *Forty-first International Conference on Machine Learning (ICML 2024, Vienna, Austria, July 21–27, 2024)*. OpenReview.net. <https://openreview.net/forum?id=pEWAccejiU2>

[37] Google. 2024. Introducing Gemini 2.0: our new AI model for the agentic era. <https://blog.google/technology/google-deepmind/google-gemini-ai-update-december-2024>

[38] Google. 2025. Gemini 2.5: Our most intelligent AI model. <https://blog.google/technology/google-deepmind/gemini-2-5-our-most-intelligent-ai-model/>

[39] MADSys group and Approaching AI. 2023. A Flexible Framework for Experiencing Cutting-edge LLM Inference Optimizations. <https://github.com/kvcache-ai/ktransformers>

ai/ktransformers

[40] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. 2024. KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization. *arXiv preprint arXiv:2401.18079* (2024).

[41] Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. 2023. Mistral 7B. *arXiv preprint arXiv:2310.06825* (2023).

[42] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibao Nong, Yulu Jia, Sun He, Hongmin Chen, Zhihao Bai, Qi Hou, Shipeng Yan, Ding Zhou, Yiyao Sheng, Zhuo Jiang, Haohan Xu, Haoran Wei, Zhang Zhang, Pengfei Nie, Leqi Zou, Sida Zhao, Liang Xiang, Zherui Liu, Zhe Li, Xiaoying Jia, Jiansi Ye, Xin Jia, and Xin Liu. 2024. MegaScale: Scaling Large Language Model Training to More Than 10,000 GPUs. <http://arxiv.org/abs/2402.15627> arXiv:2402.15627 [cs].

[43] Norm Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, Clifford Young, Xiang Zhou, Zongwei Zhou, and David A Patterson. 2023. TPU v4: An Optically Reconfigurable Supercomputer for Machine Learning with Hardware Support for Embeddings. In *Proceedings of the 50th Annual International Symposium on Computer Architecture (Orlando, FL, USA) (ISCA '23)*. Association for Computing Machinery, New York, NY, USA, Article 82, 14 pages. <https://doi.org/10.1145/3579371.3589350>

[44] Hao Kang, Qingru Zhang, Souvik Kundu, Geonhwa Jeong, Xiaozing Liu, Tushar Krishna, and Tuo Zhao. 2024. GEAR: An Efficient KV Cache Compression Recipe for Near-Lossless Generative Inference of LLM. *arXiv:2403.05527* [cs.LG]

[45] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling Laws for Neural Language Models. *CoRR* abs/2001.08361 (2020). <https://arxiv.org/abs/2001.08361>

[46] John Kim, Wiliam J. Dally, Steve Scott, and Dennis Ahts. 2008. Technology-Driven, Highly-Scalable Dragonfly Topology. In *2008 International Symposium on Computer Architecture*. 77–88. <https://doi.org/10.1109/ISCA.2008.19>

[47] Vijay Anand Korthikanti, Jared Casper, Sangkug Lynn, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. 2023. Reducing activation recomputation in large transformer models. *Proceedings of Machine Learning and Systems* 5 (2023).

[48] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2024. EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21–27, 2024*. OpenReview.net. <https://openreview.net/forum?id=1NdN7cXyB4>

[49] Heng Liao, Bingyang Liu, Xianping Chen, Zhigang Guo, Chuanning Cheng, Jianbing Wang, Xiangyu Chen, Peng Dong, Rui Meng, Wenjie Liu, Zhe Zhou, Ziyang Zhang, Yuhang Gai, Cunle Qian, Yi Xiong, Zhongyu Cheng, Jing Xia, Yuli Ma, Xi Chen, Wenhua Du, Shishong Xiao, Chunguang Li, Yong Qin, Luidong Xiang, Zhou Yu, Lv Chen, Lei Chen, Buyun Wang, Pei Wu, Junen Gao, Xiaochi Li, Jian He, Shizhuan Yan, and Bill McColl. 2025. UB-Mesh: a Hierarchically Localized nD-FullMesh Datacenter Network Architecture. *arXiv:2503.20377* [cs.AR]. <https://arxiv.org/abs/2503.20377>

[50] Sean Lie. 2022. Cerebras Architecture Deep Dive: First Look Inside the HW/SW Co-Design for Deep Learning: Cerebras Systems. In *2022 IEEE Hot Chips 34 Symposium (HCS)*. 1–34. <https://doi.org/10.1109/HCS5958.2022.9895479>

[51] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. In *MLSys*.

[52] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhao Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. 2024. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. *arXiv preprint arXiv:2402.02750* (2024).

[53] Junjie Luo, Weizhi Zhang, Ye Yuan, Yuesheng Zhao, Junwei Yang, Yiyang Gu, Bohan Wu, Binqi Chen, Ziyue Qiao, Qingqing Long, Rongcheng Xu, Xiao Luo, Wei Ju, Zhiping Xiao, Yifan Wang, Meng Xiao, Chenwu Liu, Jingyuan Yuan, Shichang Zhang, Yiqiao Jin, Fan Zhang, Xian Wu, Hangzhou Zhang, Dacheng Tao, Philip S. Yu, and Ming Zhang. 2025. Large Language Model Agent: A Survey on Methodology, Applications and Challenges. *arXiv preprint arXiv:2503.21460* (2025).

[54] Karthik Mandakolathur and Sylvain Jeanguy. 2022. Doubling all2all Performance with NVIDIA Collective Communication Library 2.12. <https://developer.nvidia.com/blog/doubling-all2all-performance-with-nvidia-collective-communication-library-2-12/>

[55] Mistral. 2024. Cheaper, Better, Faster, Stronger: Continuing to push the frontier of AI and making it accessible to all. <https://mistral.ai/news/mistral-8x22b>

[56] Dheevatsa Mudigere, Yuchen Hao, Jianyu Huang, Zhihao Jia, Andrew Tulloch, Srinivas Sridharan, Xing Liu, Mustafa Ozdal, Jade Nie, Jongsoo Park, Liang Luo, Ji Amy Yang, Leon Gao, Aarti Basu, Yuxi Hu, Yixuan Yang, Ehsan K. Ardeshani, Xiaodong Wang, Rakesh Komuravelli, Chung-Hsing Chu, Serhat Yilmaz, Huayu Li, Jiyan Qian, Zhuobo Feng, Yinbin Ma, Junjie Yang, Ellie

ISCA '25, 2025 年 6 月 21–25 日, 日本越前町, 邓承祺, 阮冲, 戴大麦, 高华作, 李佳实, 张立越, 黄盼盼, 周上炎, 马世荣, 梁文锋, 何颖, 王雨晴, 刘宇轩, and Y.X. Wei

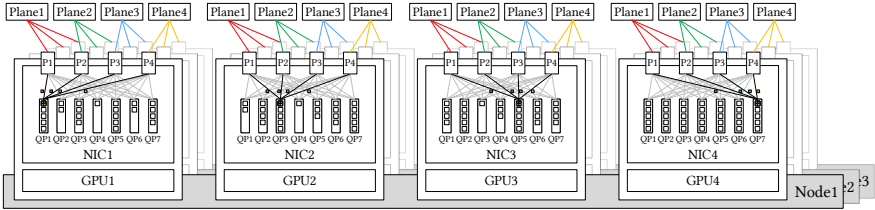


Figure 4: 理想的多平面网络：每个网卡配备多个物理端口，每个端口连接到不同的网络平面。单个队列对（QP）可以同时利用所有可用端口发送和接收数据包，这需要网卡原生支持乱序放置。

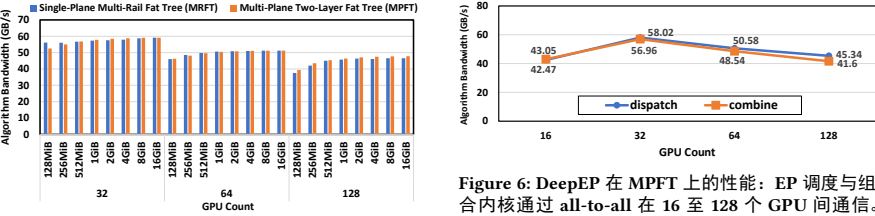


Figure 5: MRFT 和 MPFT 网络在 32 至 128 个 GPU 上的 NCCL all-to-all 性能。

如图 5 所示，多平面网络的 All-to-All 性能与单平面多轨网络非常相似。这种性能相当性可归因于 NCCL 的 PXN [54] 机制，该机制通过 NVLink 优化了多轨拓扑中的流量转发。多平面拓扑同样受益于此机制。如图 5.1.1 所示，在 16 个 GPU 上进行的 All-to-All 通信测试结果显示，MPFT 与 MRFT 拓扑之间的延迟差异微乎其微。

为了评估 MPFT 在实践训练场景中的 All-to-All 通信性能，我们测试了训练中常用的 EP 通信模式。如图 6 所示，在多平面网络中，每个 GPU 都实现了超过 40GB/s 的高带宽，提供了满足训练需求的可靠性能。

2. Training Throughput for DeepSeek-V3 Model: 我们还在表 4 中比较了 DeepSeek-V3 模型在 MPFT 和 MRFT 下的训练指标。MFU（模型浮点运算利用率）是基于 BF16 峰值性能计算的。因果 MFU 仅考虑注意力矩阵下三角部分的浮点运算（与 FlashAttention[19, 20] 一致），而非因果 MFU 则包含

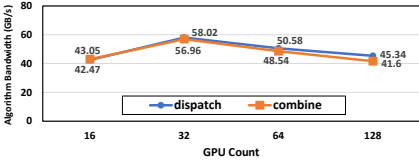


Figure 6: DeepEP 在 MPFT 上的性能：EP 调度与组合内核通过 all-to-all 在 16 至 128 个 GPU 间通信。每个 GPU 处理 4096 个令牌。观测到的吞吐量几乎饱和了 400Gbps 的 NIC 带宽。

整个注意力矩阵的浮点运算（与 Megatron [47] 一致）。1F、1B 和 1W 分别表示前向时间、输入反向时间和权重反向时间。

5.2 Low Latency Networks

在我们的模型推理中，大规模 EP 严重依赖于 all-to-all 通信，这对带宽和延迟都高度敏感。考虑第 2.3.2 节讨论的典型场景，在 50GB/s 的网络带宽下，数据传输理想情况下应耗时约 120 μ s。因此，微秒量级的固有网络延迟会严重影响系统性能，其影响不可忽视。

5.2.1 1B or RoCE. 如表 5 所示，1B 始终能实现更低的延迟，使其成为分布式训练和推理等延迟敏感型工作负载的首选。尽管 1B 与基于融合以太网的 RDMA（RoCE）相比具有更优的延迟性能，但它也存在一些局限性：

Wen, Hong Li, Lin Yang, Chonglin Sun, Whitney Zhao, Dmitry Melts, Krishna Dhulipala, K. R. Kishore, Tyler Graf, Assaf Eisenman, Kiran Kumar Matam, Adi Gangidi, Guoqiang Jerry Chen, Manoj Krishnan, Avinash Nayak, Krishnakumar Nair, Bharath Muthiah, Mahmood khorasadi, Pallab Bhattacharya, Petr Lapukhov, Maxim Naumov, Ajit Mathews, Lin Qiao, Mikhail Smelyanskiy, Bill Jia, and Vijay Rao. 2023. Software-Hardware Co-design for Fast and Scalable Training of Deep Learning Recommendation Models. <http://arxiv.org/abs/2104.05158> arXiv:2104.05158 [cs].

[57] NVIDIA. 2022. Improving Network Performance of HPC Systems Using NVIDIA Magnum IO NVSHMEM and GPUDirect Async. <https://developer.nvidia.com/blog/improving-network-performance-of-hpc-systems-using-nvidia-magnum-io-nvshmem-and-gpudirect-async/>

[58] NVIDIA. 2025. NVIDIA DGX Spark: A Grace Blackwell AI supercomputer on your desk. <https://www.nvidia.com/en-us/products/workstations/dgx-spark/>

[59] OpenAI. 2024. Hello GPT-4o. <https://openai.com/index/hello-gpt-4o/>

[60] OpenAI. 2024. Introducing OpenAI o1. <https://openai.com/o1/>

[61] OpenAI. 2025. Introducing OpenAI o3 and o4-mini. <https://openai.com/index/introducing-o3-and-o4-mini/>

[62] Kun Qian, Yongqing Xi, Jiamin Cao, Jiaqi Gao, Yichi Xu, Yu Guan, Binzhang Fu, Xuemei Shi, Fangbo Zhu, Rui Miao, Chao Wang, Peng Wang, Pengcheng Zhang, Xianlong Zeng, Eddie Ruan, Ziping Yao, Ennan Zhai, and Dennis Cai. 2024. Aihaba HPC: A Data Center Network for Large Language Model Training. In *Proceedings of the ACM SIGCOMM 2024 Conference* (Sydney, NSW, Australia) (ACM SIGCOMM '24). Association for Computing Machinery, New York, NY, USA, 691–706. <https://doi.org/10.1145/3651890.3672265>

[63] Zhen Qin, Weigao Sun, Dong Li, Xuyang Shen, Weixuan Sun, and Yiran Zhong. 2024. Various lengths, constant speed: efficient language modeling with lightning attention. In *Proceedings of the 41st International Conference on Machine Learning* (Vienna, Austria) (ICML '24). JMLR.org, Article 1688, 19 pages.

[64] Rafad Rafailow, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2024. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. arXiv:2305.18290 [cs.LG] <https://arxiv.org/abs/2305.18290>

[65] Md Shafayat Rahman, Saptarshi Bhowmik, Yevgeniy Ryasianskiy, Xin Yuan, and Michael Lang. 2019. Topology-custom UGAL routing on dragonfly. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (Denver, Colorado) (SC '19). Association for Computing Machinery, New York, NY, USA, Article 17, 15 pages. <https://doi.org/10.1145/3295500.3356208>

[66] Bitā Darvish Rouhani, Ritchie Zhao, Ankī More, Mathew Hall, Alireza Khodamoradi, Summer Deng, Dhruv Choudhary, Marius Cornea, Eric Dellinger, Kristof Denolf, Stosic Dusan, Vennugol Elango, Maximilian Golub, Alexander Heinecke, Phil James-Roxby, Dharmesh Jani, Gaurav Kolhe, Martin Langhammer, Ada Li, Levi Melnick, Maral Mesmakhosroshahi, Andres Rodriguez, Michael Schulte, Rasoul Shafipour, Lei Shao, Michael Sit, Pradeep Dubey, Paulius Micikevicius, Maxim Naumov, Colin Verrilli, Ralph Wittig, Doug Burger, and Eric Chung. 2023. Microscaling Data Formats for Deep Learning. arXiv:2310.10537 [cs.LG] <https://arxiv.org/abs/2310.10537>

[67] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. arXiv:1707.06347 [cs.LG] <https://arxiv.org/abs/1707.06347>

[68] ByteDance Seed. 2025. Seed1.5-Thinking: Advancing Superb Reasoning Models with Reinforcement Learning. arXiv:2504.13914 [cs.CL] <https://arxiv.org/abs/2504.13914>

[69] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. DeepSeek-Math: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300 [cs.CL] <https://arxiv.org/abs/2402.03300>

[70] Noam Shazeer. 2019. Fast Transformer Decoding: One Write-Head is All You Need. CoRR abs/1911.02150 (2019). <http://arxiv.org/abs/1911.02150>

[71] Qwen Team. 2024. Qwen2.5 Technical Report. arXiv preprint arXiv:2412.15115 (2024).

[72] Qwen Team. 2025. Qwen3: Think Deeper, Act Faster. <https://github.com/QwenLM/Qwen3>

[73] Song Wang, Bing Yu, Wenwu Xiao, Fujian Bai, Xiaodong Long, Liang Bai, Xuerong Jia, Fengguo Zuo, Jie Tan, Yixin Guo, Peng Sun, Jun Zhou, Qiong Zhan, Sheng Hu, Yu Zhou, Yi Kang, Qiwei Ren, and Xiping Jiang. 2023. A 135 GBps/Gbit 0.66 pJ/bit Stacked Embedded DRAM with Multilayer Arrays by Fine Pitch Hybrid Bonding and Mini-TSV. In *2023 IEEE Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits)* 1–2. <https://doi.org/10.23919/VLSITechnologyandCir57934.2023.10185427>

[74] xAI. 2024. Grok-2 Beta Release. <https://x.ai/news/grok-2>.

[75] xAI. 2024. Our Gigafactory of Compute-Colossus. <https://x.ai/colossus>.

[76] Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, Y. X. Wei, Lean Wang, Ziping Xiao, Yuqing Wang, Chong Ruan, Ming Zhang, Wenfeng Liang, and Wangding Zeng. 2025. Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention. <https://arxiv.org/abs/2502.11089>

[77] Jintao Zhang, Haofeng Huang, Pengle Zhang, Jia Wei, Jun Zhu, and Jianfei Chen. 2025. SageAttention2: Efficient Attention with Thorough Outlier Smoothing and Per-thread INT4 Quantization. arXiv:2411.10958 [cs.LG] <https://arxiv.org/abs/2411.10958>

[78] Chenggang Zhao, Liang Zhao, Jiashi Li, and Zhean Xu. 2025. DeepGEMM: clean and efficient FPG GEMM kernels with fine-grained scaling. <https://github.com/deepseek-ai/DeepGEMM>.

[79] Chenggang Zhao, Shangyan Zhou, Liye Zhang, Chengqi Deng, Zhean Xu, Yuxuan Liu, Kuai Yu, Jiashi Li, and Liang Zhao. 2025. DeepEP: an efficient expert-parallel communication library. <https://github.com/deepseek-ai/DeepEP>.

[80] Size Zheng, Jin Fang, Xuegui Zheng, Qi Hou, Wenlei Bao, Ningxin Zheng, Ziheng Jiang, Dongyang Wang, Jianxi Ye, Haibin Lin, Li-Wen Chang, and Xin Liu. 2025. TileLink: Generating Efficient Compute-Communication Overlapping Kernels using Tile-Centric Primitives. arXiv:2503.20313 [cs.DC] <https://arxiv.org/abs/2503.20313>

[81] Yimin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, Santa Clara, CA, 193–210. <https://www.usenix.org/conference/osdi24/presentation/zhong-yimin>

Table 4: MPFT 与 MRFT 网络之间的训练指标对比。

Metric	MPFT	MRFT
tokens/day (B)	272.80	272.52
time/step (s)	19.926	19.946
1F (s)	1.13	1.13
bubble (s)	2.06	2.03
1B (s)	1.99	1.99
1W (s)	0.48	0.48
1F1B (s)	13.95	14.00
TFLOPS (non-causal)	432	432
TFLOPS (causal)	385	385
MFU (non-causal)	43.73%	43.68%
MFU (causal)	38.94%	38.90%

Table 5: IB、RoCE 与节点内 NVLink 之间传输 64B 数据的 CPU 端到端延迟对比。

Link Layer	Same Leaf	Cross Leaf
RoCE	3.6us	5.6us
InfiniBand	2.8us	3.7us
NVLink	3.33us	-

5.2.2 Recommendations for RoCE Improvements. 虽然 RoCE 有潜力成为 IB 的一种经济高效的替代方案，但其目前在延迟和可扩展性方面的局限性，使其无法完全满足大规模 AI 系统的需求。下面，我们概述了改进 RoCE 的具体建议：

我们建议以太网供应商开发专门针对 RDMA 工作负载优化的 RoCE 交换机，移除不必要的以太网功能。Slingshot 架构 [22] 展示了基于以太网的设计如何能达到与 IB 相当的延迟性能。同样，博通 [13] 近期的创新，包括 AI 转发头 (AIFH) 和即将推出的低延迟以太网交换机，证明了为 AI 量身定制高性能以太网结构的可行性。我们期待在这一方向上持续创新。当前的 RoCE 交换机仅支持有限数量的优先级队列，这对于涉及并发通信模式（如专家并行的全对全和数据并行的全规约）的复杂 AI 工作负载来说是不够的。在此类混合工作负载中，全对全流量可能因突发性的多对一传输而导致入口拥塞，从而可能降低整体网络性能。为了解决入口拥塞对其他流量的影响，一种方法是采用虚拟输出队列（VOQ），为每个 QP 分配一个专用的虚拟队列以隔离流量。或者，可以采用更有效的拥塞控制（CC）机制，如基于 RTT 的 CC（RTTCC）或用户可编程 CC（PCC），

从而实现 NIC 与交换机的协同优化，在动态流量条件下保持低延迟和高吞吐量。

第 3 章：方法本章描述了用于评估所提出方法的数据集、实验设置和评估指标。本章的结构如下。第 3.1 节描述了用于评估的数据集。第 3.2 节描述了实验设置。第 3.3 节描述了评估指标。第 3.4 节描述了基线方法。第 3.5 节描述了所提出的方法。方法 本章描述了用于评估所提出方法的数据集、实验设置和评估指标。本章的结构如下。第 6 节描述了用于评估的数据集。第 7 节描述了实验设置。第 8 节描述了评估指标。第 9 节描述了基线方法。第 10 节描述了所提出的方法。

第 3.1 节：数据集本节描述了用于评估的数据集。数据集 A 是一个公开可用的数据集，包含 1000 个样本。数据集 B 是一个私有数据集，包含 500 个样本。数据集 C 是一个合成数据集，包含 2000 个样本。

6 数据集

本节描述了用于评估的数据集。我们使用了三个数据集：数据集 A、数据集 B 和数据集 C。数据集 A 是一个公开可用的数据集，包含 1000 个样本。数据集 B 是一个私有数据集，包含 500 个样本。数据集 C 是一个合成数据集，包含 2000 个样本。第 3.2 节：实验设置我们使用了 5 折交叉验证来评估模型性能。我们使用了 80% 的数据进行训练，20% 的数据进行测试。我们使用了 Adam 优化器，学习率为 0.001。

7 实验设置

本节描述了实验设置。我们使用了 5 折交叉验证来评估模型性能。我们使用了 80% 的数据进行训练，20% 的数据进行测试。我们使用了 Adam 优化器，学习率为 0.001。第 3.3 节：评估指标我们使用了准确率、精确率、召回率和 F1 分数来评估模型性能。精确率定义为真正例数除以真正例数与假正例数之和。F1 分数定义为精确率和召回率的调和平均数。

8 评估指标

本节描述了评估指标。我们使用了准确率、精确率、召回率和 F1 分数来评估模型性能。准确率定义为正确预测的样本数除以总样本数。精确率定义为真正例数除以真正例数与假正例数之和。召回率定义为真正例数除以真正例数与假负例数之和。F1 分数定义为精确率和召回率的调和平均数。

我们使用了三种基线方法：方法 A、方法 B 和方法 C。方法 A 是一种基于规则的方法。方法 B 是一种基于机器学习的方法。

9 基线方法

本节描述了基线方法。我们使用了三种基线方法：方法 A、方法 B 和方法 C。方法 B 是一种基于机器学习的方法。方法 C 是一种基于深度学习的方法。
第 3.5 节：所提出的方法它由三个主要组件组成：组件 A、组件 B 和组件 C。组件 C 是一个分类器。

10 所提出的方法

本节描述了所提出的方法。所提出的方法是一种基于深度学习的方法。组件 A 是一个特征提取器。组件 B 是一个注意力机制。组件 C 是一个分类器。

10.0.1 *InfiniBand GPU Direct Async (IBGDA)*. 我们利用 IBGDA [2, 57] 来降低网络通信的延迟。传统上，网络通信涉及创建一个 CPU 代理线程：一旦 GPU 准备好数据，它必须通知 CPU 代理，然后由该代理填充工作请求（WR）的控制信息，并通过门铃机制通知 NIC 以启动数据传输。这个过程引入了额外的通信开销。

IBGDA 通过允许 GPU 直接填充 WR 内容并写入 RDMA 门铃 MMIO 地址来解决此问题。通过在 GPU 内管理整个控制平面，IBGDA 消除了与 GPU-CPU 通信相关的显著延迟开销。此外，当发送大量小数据包时，控制平面处理器很容易成为瓶颈。由于 GPU 拥有多个并行线程，发送方可以利用这些线程来分配工作负载，从而避免此类瓶颈。一系列工作——包括

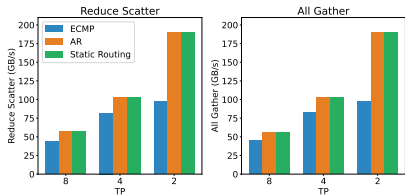


Figure 7: 不同路由方法（ECMP、AR、静态路由）和 TP 维度下，AllGather 和 ReduceScatter 通信原语的 RoCE 网络带宽。

我们的 DeepEP [79]——已经利用了 IBGDA 并报告了显著的性能提升 [1, 15, 80]。因此，我们主张此类功能应在加速器设备中得到广泛支持。

11 Discussion and Insights for Future Hardware Architecture Design

基于前面各节的讨论，我们总结了关键的架构见解，并概述了针对大规模 AI 工作负载量身定制的硬件设计的未来方向。

第 2.3.2 节强调了大规模纵向扩展网络对于加速模型推理的重要性。第 3 节讨论了高效支持低精度计算和通信的必要性。第 4 节探讨了纵向扩展和横向扩展架构的融合，以及几项提出的增强措施。第 5 节专注于多平面网络拓扑，并确定了基于以太网的互连所需的关键改进。

这些章节共同在具体的应用场景中识别了硬件限制，并提供了相应的建议。在此基础上，本节将讨论扩展到更广泛的考量，并为未来的硬件架构设计提出了为了减轻与静默数据损坏相关的风险，硬件必须纳入超越传统 ECC 的先进错误检测机制。基于校验和的验证或硬件加速的冗余检查等技术可以为大规模部署提供更高的可靠性。

此外，硬件供应商应向最终用户提供全面的诊断工具包，使他们能够严格验证其系统的完整性，并主动识别任何潜在的静默数据损坏。此类工具包作为标

准硬件包的一部分嵌入，可促进透明度，并能在整个运行生命周期内实现持续验证，从而增强整体系统的可信度。

11.1 CPU Bottlenecks and Interconnects

尽管加速器设计常常占据中心舞台，但 CPU 对于协调计算、管理 I/O 以及维持系统吞吐量仍然至关重要。然而，当前的架构面临着几个关键瓶颈：首先，如第4.5节所述，CPU 与 GPU 之间的 PCIe 接口常常成为带宽瓶颈，尤其是在大规模参数、梯度或 KV 缓存传输期间。为了解决这一问题，未来的系统应采用直接的 CPU-GPU 互连——例如 NVLink 或 Infinity Fabric——或者将 CPU 和 GPU 都集成到纵向扩展域中，从而消除节点内的瓶颈。除了 PCIe 的限制之外，维持如此高的数据传输速率还需要极高的内存带宽。例如，要饱和 PCIe 5.0 的 160 条通道，每个节点需要超过 640 GB/s 的带宽，这转化为每个节点大约 1 TB/s 的内存带宽需求——这对传统的 DRAM 架构构成了重大挑战。最后，对延迟敏感的任务，如内核启动和网络处理，需要高的单核 CPU 性能，通常要求基础频率高于 4 GHz。此外，现代 AI 工作负载需要每个 GPU 配备足够的 CPU 核心，以防止控制端出现瓶颈。对于基于芯粒的架构，需要额外的核心来支持缓存感知的工作负载分区和隔离。

11.2 Toward Intelligent Networks for AI

为了满足延迟敏感型工作负载的需求，未来的互连技术必须优先考虑低延迟和智能网络：

11.3 Discussion on Memory-Semantic Communication and Ordering Issue

使用加载/存储内存语义的节点间通信是高效且对程序员友好的，但当前的实现受到内存排序挑战的阻碍。例如，写入数据后，发送方必须在更新标志以通知接收方之前发出显式的内存屏障（fence），以确保数据一致性。这种严格的排序引入了额外的往返时间（RTT）延迟，并可能阻塞发出指令的线程，阻碍正在进行的存储操作并降低吞吐量。类似的无序同步问题也出现在消息语义的 RDMA 中；例如，在 InfiniBand 或 NVIDIA BlueField-3 上进行常规 RDMA 写操作后，执行带有数据包喷洒的 RDMA 原子加操作可能会产生额外的 RTT 延迟。为了解决这些问题，我们主张硬件应支持为内存语义通信提供内置的排序保证。这种一致性应在编程层面（例如，通过获取/释放语义）和接收端的硬件层面强制执行，从而实现无需额外开销的顺序交付。有几种可能的方法。例如，接收方可以缓冲原子消息并使用数据包序列号（PSN）来确保顺序处理；这种方法简单直接，能有效保持正确性。或者，基于区域的获取/释放（RAR）机制尤其具有吸引力，因为它能为远程内存访问实现真正的获取/释放语义，并为一系列工作负载提供更大的灵活性。在这种方法中，接收端硬件维护轻量级元数据——例如位图或基于区域的计数器——来跟踪内存区域的状态。获取和释放操作限定在特定的地址范围内，从而实现高效、硬件强制的排序，而无需发送端的显式屏障。值得注意的是，这两种方法都适合在 NIC 或 I/O 芯片上实现，并且适用于内存语义和消息语义的 RDMA 原语，从而拓宽了它们的实际应用范围。

11.4 In-Network Computation and Compression

EP 涉及两个关键的 all-to-all 阶段——**dispatch** 和 **combine**——这为网内优化提供了重要机会。**dispatch** 阶段类似于一个小规模的多播操作，其中单个消息必须转发到多个目标设备。一个能够在硬件层面实现数据包自动复制并转发到多个目的地的协议，可以极大地减少通信开销并提高效率。**combine** 阶段作为一个小规模的规定操作，可以从网内聚合技术中受益。然而，由于 EP 组合中规定范围小且工作负载不均衡，以灵活的方式实现网内聚合具有挑战性。

此外，如第3.2节所述，LogFMT 能够以对模型性能影响最小的方式传输低精度令牌。将 LogFMT 原生地集成到网络硬件中，可以通过增加熵密度和减少带宽使用来进一步优化通信。硬件加速的压缩和解压将使 LogFMT 能够无缝集成到分布式系统中，从而提高整体吞吐量。

11.5 Memory-Centric Innovations

11.5.1 Limitations of Memory Bandwidth. 模型规模的指数级增长已经超过了高带宽内存（HBM）技术的进步。这种差距造成了内存瓶颈，特别是在像 Transformer 这样注意力密集的结构中。

11.5.2 Suggestions:

12 Conclusion

DeepSeek-V3 体现了软硬件协同设计在提升大规模 AI 系统的可扩展性、效率和鲁棒性方面的变革潜力。通过解决当前硬件架构的局限性并提出可行的建议，本文为下一代 AI 优化硬件提供了路线图。随着 AI 工作负载在复杂性和规模上持续增长，这些创新对于推动智能系统的未来至关重要。

References

[1] Elena Agostini, Davide Rossetti, and Sreeram Potluri. 2017. Offloading Communication Control Logic in GPU Accelerated Applications. In *2017 17th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGRID)*. 248–257. <https://doi.org/10.1109/CCGRID.2017.29>

[2] E. Agostini, D. Rossetti, and S. Potluri. 2018. GPUDirect Async: Exploring GPU synchronous communication techniques for InfiniBand clusters. *J. Parallel and Distrib. Comput.* 114 (2018), 28–45. <https://doi.org/10.1016/j.jpdc.2017.12.007>

[3] AI@Meta. 2024. Llama 3 Model Card. https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md

[4] AI@Meta. 2024. Llama 3.1 Model Card. https://github.com/meta-llama/llama-models/blob/main/models/llama3.1/MODEL_CARD.md

[5] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. 2023. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *arXiv preprint arXiv:2305.13245* (2023).

[6] AMD. 2025. AMD Ryzen AI Max+ PRO 395: Designed to power a new generation of compact Copilot+ PC workstations. <https://www.amd.com/en/products/processors/laptop/ryzen-pro-ai-max-pro-300-series/amd-ryzen-ai-max-plus-pro-395.html>

[7] Wei An, Xiao Bi, Guanting Chen, Shanhuang Chen, Chengqi Deng, Honghui Ding, Kai Dong, Qiushi Du, Wenjun Gao, Kang Guan, Jianzhong Guo, Yongqiang Guo, Zhe Fu, Ying He, Panpan Huang, Jiashi Li, Wenfeng Liang, Xiaodong Liu, Xin Liu, Yiyuan Liu, Yuxuan Liu, Shanghao Lu, Xuan Lu, Xiaotao Nie, Tian Pei, Junjie Qiu, Hui Qu, Zehui Ren, Zhangli Sha, Xuecheng Su, Xiaowen Sun, Yixuan Tan, Minghui Tang, Shiyu Wang, Yaohui Wang, Yongji Wang, Ziwei Xie, Yiliang Xiong, Yanhong Xu, Shengfeng Ye, Shuiping Yu, Yukun Zha, Liyue Zhang, Haowei Zhang, Mingchuan Zhang, Wentao Zhang, Yichao Zhang, Chenggang Zhao, Yao Zhao, Shangyan Zhou, Shunfeng Zhou, and Yuheng Zou. 2024. Fire-Flyer AI-HPC: A Cost-Effective Software-Hardware Co-Design for Deep Learning. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–23. <https://doi.org/10.1109/SC41406.2024.00089>

[8] Anthropic. 2024. Claude 3.5 Sonnet. <https://www.anthropic.com/news/claude-3-5-sonnet>

[9] Anthropic. 2025. Claude 3.7 Sonnet and Claude Code. <https://www.anthropic.com/news/claude-3-7-sonnet>

[10] Apple. 2024. Apple introduces M4 Pro and M4 Max. <https://www.apple.com/newsroom/2024/10/apple-introduces-m4-pro-and-m4-max/>

[11] Iz Beltagy, Matthew E. Peters, and Arman Cohan. 2020. Longformer: The Long-Document Transformer. *arXiv:2004.05150* (2020).

[12] Nils Blach, Maciej Besta, Daniele De Sensi, Jens Domke, Hussein Harake, Shigang Li, Patrick Iff, Marek Konieczny, Kartik Lakhotia, Ales Kubicek, Marcel Ferrari, Fabrizio Petrini, and Torsten Hoefler. 2025. A high-performance design, implementation, deployment, and evaluation of the slim fly network. In *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation* (Santa Clara, CA, USA) (NSDI'24). USENIX Association, USA, Article 57, 20 pages.

[13] Broadcom. 2025. Scale Up Ethernet Framework. <https://docs.broadcom.com/doc/scale-up-ethernet-framework>

- [14] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net. <https://openreview.net/forum?id=PEpbUobJfv>
- [15] Shaoyuan Chen, Wencong Xiao, Yutong Lin, Mingxing Zhang, Yingdi Shan, Jinlei Jiang, Kang Chen, and Yongwei Wu. 2025. Efficient Heterogeneous Large Language Model Decoding with Model-Attention Disaggregation. *arXiv:2405.01814* [cs.LG] <https://arxiv.org/abs/2405.01814>
- [16] ULTRA ACCELERATOR LINK CONSORTIUM. 2025. Introducing UALink 200G 1.0 Specification. https://ualinkconsortium.org/wp-content/uploads/2025/04/UALink-1.0-White_Paper_FINAL.pdf
- [17] Ultra Ethernet Consortium. 2023. Overview of and Motivation for the Forthcoming Ultra Ethernet Consortium Specification. <https://ultraethernet.org/wp-content/uploads/sites/20/2023/10/23.07.12-UEC-1.0-Overview-FINAL-WITH-LOGO.pdf>
- [18] Ultra Ethernet Consortium. 2024. UEC Progresses Towards v1.0 Set of Specifications. <https://ultraethernet.org/uec-progresses-towards-v1-0-set-of-specifications/>
- [19] Tri Dao. 2023. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning.
- [20] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems*.
- [21] Tri Dao and Albert Gu. 2024. Transformers are SSMs: generalized models and efficient algorithms through structured state space duality. In *Proceedings of the 41st International Conference on Machine Learning (Vienna, Austria) (ICML '24)*. JMLR.org, Article 399, 31 pages.
- [22] Daniele De Sensi, Salvatore Di Girolamo, Kim H. McMahon, Duncan Roweth, and Torsten Hoefler. 2020. An In-Depth Analysis of the Slingshot Interconnect. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–14. <https://doi.org/10.1109/SC41405.2020.00039>
- [23] DeepSeek-AI. 2024. DeepSeek-Coder-V2: Breaking the Barrier of Closed-Source Models in Code Intelligence. *CoRR* abs/2406.11931 (2024). <https://doi.org/10.48550/arXiv.2406.11931>
- [24] DeepSeek-AI. 2024. DeepSeek LLM: Scaling Open-Source Language Models with Longtermism. *CoRR* abs/2401.02954 (2024). <https://doi.org/10.48550/arXiv.2401.02954>
- [25] DeepSeek-AI. 2024. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. *CoRR* abs/2405.04434 (2024). <https://doi.org/10.48550/arXiv.2405.04434>
- [26] DeepSeek-AI. 2024. DeepSeek-V3 Technical Report. (2024). *arXiv:2412.19437* [cs.CL] <https://arxiv.org/abs/2412.19437>
- [27] DeepSeek-AI. 2024. DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models. *CoRR* abs/2401.06066 (2024). <https://doi.org/10.48550/arXiv.2401.06066>
- [28] DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv:2501.12948* [cs.CL] <https://arxiv.org/abs/2501.12948>
- [29] DeepSeek-AI. 2025. DualPipe: A bidirectional pipeline parallelism algorithm for computation-communication overlap in V3/R1 training. <https://github.com/deepseek-ai/dualpipe>.
- [30] DeepSeek-AI. 2025. Fire-Flyer File System. <https://github.com/deepseek-ai/3FS>
- [31] DeepSeek-AI. 2025. Profiling Data in DeepSeek Infra. <https://github.com/deepseek-ai/profile-data?tab=readme-ov-file#inference>
- [32] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2022. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323* (2022).
- [33] Adithya Gangidi, Rui Miao, Shengbao Zheng, Sai Jayesh Bondu, Guilherme Goes, Hany Morsy, Rohit Puri, Mohammad Rifadi, Ashmitha Jeevaraj Shetty, Jingyi Yang, Shuqiang Zhang, Mikel Jimenez Fernandez, Shashidhar Gandham, and Hongyi Zeng. 2024. RDMA over Ethernet for Distributed Training at Meta Scale. In *Proceedings of the ACM SIGCOMM 2024 Conference (Sydney, NSW, Australia) (ACM SIGCOMM '24)*. Association for Computing Machinery, New York, NY, USA, 57 – 70. <https://doi.org/10.1145/3651890.3672233>
- [34] Patrick Geffray and Torsten Hoefler. 2008. Adaptive Routing Strategies for Modern High Performance Networks. In *2008 16th IEEE Symposium on High Performance Interconnects*. 165–172. <https://doi.org/10.1109/HOTI.2008.21>
- [35] Amir Gholami, Zhewei Yao, Sehoon Kim, Coleman Hooper, Michael W. Mahoney, and Kurt Keutzer. 2024. AI and Memory Wall. *IEEE Micro* 44, 03 (May 2024), 33–39. <https://doi.org/10.1109/MM.2024.3373763>
- [36] Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. 2024. Better & Faster Large Language Models via Multi-token Prediction. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net. <https://openreview.net/forum?id=pEWAcejiU2>
- [37] Google. 2024. Introducing Gemini 2.0: our new AI model for the agentic era. <https://blog.google/technology/google-deepmind/google-gemini-ai-update-december-2024>
- [38] Google. 2025. Gemini 2.5: Our most intelligent AI model. <https://blog.google/technology/google-deepmind/gemini-model-thinking-updates-march-2025/>
- [39] MADSys group and Approaching AI. 2025. A Flexible Framework for Experiencing Cutting-edge LLM Inference Optimizations. <https://github.com/kvcache-ai/ktransformers>
- [40] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. 2024. KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization. *arXiv preprint arXiv:2401.18079* (2024).
- [41] Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. 2023. Mistral 7B. *arXiv preprint arXiv:2310.06825* (2023).
- [42] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibiao Nong, Yulu Jia, Sun He, Hongmin Chen, Zhihao Bai, Qi Hou, Shipeng Yan, Ding Zhou, Yiyao Sheng, Zhuo Jiang, Haohan Xu, Haoran Wei, Zhang Zhang, Pengfei Nie, Leqi Zou, Sida Zhao, Liang Xiang, Zherui Liu, Zhe Li, Xiaoying Jian, Jianxi Ye, Xin Jin, and Xin Liu. 2024. MegaScale: Scaling Large Language Model Training to More Than 10,000 GPUs. <http://arxiv.org/abs/2402.15627> [cs].

- [43] Norm Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, Clifford Young, Xiang Zhou, Zongwei Zhou, and David A Patterson. 2023. TPU v4: An Optically Reconfigurable Supercomputer for Machine Learning with Hardware Support for Embeddings. In *Proceedings of the 50th Annual International Symposium on Computer Architecture* (Orlando, FL, USA) (ISCA '23). Association for Computing Machinery, New York, NY, USA, Article 82, 14 pages. <https://doi.org/10.1145/3579371.3589350>
- [44] Hao Kang, Qingru Zhang, Souvik Kundu, Geonhwa Jeong, Zaoxing Liu, Tushar Krishna, and Tuo Zhao. 2024. GEAR: An Efficient KV Cache Compression Recipe for Near-Lossless Generative Inference of LLM. *arXiv:2403.05527* [cs.LG]
- [45] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling Laws for Neural Language Models. *CoRR abs/2001.08361* (2020). *arXiv:2001.08361* <https://arxiv.org/abs/2001.08361>
- [46] John Kim, William J. Dally, Steve Scott, and Dennis Abts. 2008. Technology-Driven, Highly-Scalable Dragonfly Topology. In *2008 International Symposium on Computer Architecture*. 77–88. <https://doi.org/10.1109/ISCA.2008.19>
- [47] Vijay Anand Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. 2023. Reducing activation recomputation in large transformer models. *Proceedings of Machine Learning and Systems* 5 (2023).
- [48] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2024. EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net. <https://openreview.net/forum?id=1NdN7cXyB4>
- [49] Heng Liao, Bingyang Liu, Xianping Chen, Zhigang Guo, Chuanning Cheng, Jianbing Wang, Xiangyu Chen, Peng Dong, Rui Meng, Wenjie Liu, Zhe Zhou, Ziyang Zhang, Yuhang Gai, Cunle Qian, Yi Xiong, Zhongwu Cheng, Jing Xia, Yuli Ma, Xi Chen, Wenhua Du, Shizhong Xiao, Chungang Li, Yong Qin, Liudong Xiong, Zhou Yu, Lv Chen, Lei Chen, Buyun Wang, Pei Wu, Junen Gao, Xiaochu Li, Jian He, Shizhuan Yan, and Bill McColl. 2025. UB-Mesh: a Hierarchically Localized nD-FullMesh Datacenter Network Architecture. *arXiv:2503.20377* [cs.AR] <https://arxiv.org/abs/2503.20377>
- [50] Sean Lie. 2022. Cerebras Architecture Deep Dive: First Look Inside the HW/SW Co-Design for Deep Learning : Cerebras Systems. In *2022 IEEE Hot Chips 34 Symposium (HCS)*. 1–34. <https://doi.org/10.1109/HCS55958.2022.9895479>
- [51] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Weichen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. In *MLSys*.
- [52] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. 2024. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. *arXiv preprint arXiv:2402.02750* (2024).
- [53] Junyu Luo, Weizhi Zhang, Ye Yuan, Yusheng Zhao, Junwei Yang, Yiyang Gu, Bohan Wu, Binqi Chen, Ziyue Qiao, Qingqing Long, Rongcheng Tu, Xiao Luo, Wei Ju, Zhiping Xiao, Yifan Wang, Meng Xiao, Chenwu Liu, Jingyang Yuan, Shichang Zhang, Yiqiao Jin, Fan Zhang, Xian Wu, Hanqing Zhao, Dacheng Tao, Philip S. Yu, and Ming Zhang. 2025. Large Language Model Agent: A Survey on Methodology, Applications and Challenges. *arXiv preprint arXiv:2503.21460* (2025).
- [54] Karthik Mandakolathur and Sylvain Jeagey. 2022. Doubling all2all Performance with NVIDIA Collective Communication Library 2.12. <https://developer.nvidia.com/blog/doubling-all2all-performance-with-nvidia-collective-communication-library-2-12/>
- [55] Mistral. 2024. Cheaper, Better, Faster, Stronger: Continuing to push the frontier of AI and making it accessible to all. <https://mistral.ai/news/mixtral-8x22b>
- [56] Dheevatsa Mudigere, Yuchen Hao, Jianyu Huang, Zhihao Jia, Andrew Tulloch, Srinivas Sridharan, Xing Liu, Mustafa Ozdal, Jade Nie, Jongsoo Park, Liang Luo, Jie Amy Yang, Leon Gao, Dmytro Ivchenko, Aarti Basant, Yuxi Hu, Jiyan Yang, Ehsan K. Ardestani, Xiaodong Wang, Rakesh Komuravelli, Ching-Hsiang Chu, Serhat Yilmaz, Huayu Li, Jiyuan Qian, Zhuobo Feng, Yinbin Ma, Junjie Yang, Ellie Wen, Hong Li, Lin Yang, Chonglin Sun, Whitney Zhao, Dimitry Melts, Krishna Dhulipala, K. R. Kishore, Tyler Graf, Asaf Eisenman, Kiran Kumar Matam, Adi Gangidi, Guoqiang Jerry Chen, Manoj Krishnan, Avinash Nayak, Krishnakumar Nair, Bharath Muthiah, Mahmoud khorashadi, Pallab Bhattacharya, Petr Lapukhov, Maxim Naumov, Ajit Mathews, Lin Qiao, Mikhail Smelyanskiy, Bill Jia, and Vijay Rao. 2023. Software-Hardware Co-design for Fast and Scalable Training of Deep Learning Recommendation Models. <http://arxiv.org/abs/2104.05158> *arXiv:2104.05158* [cs].
- [57] NVIDIA. 2022. Improving Network Performance of HPC Systems Using NVIDIA Magnum IO NVSHMEM and GPUDirect Async. <https://developer.nvidia.com/blog/improving-network-performance-of-hpc-systems-using-nvidia-magnum-io-nvshmem-and-gpudirect-async/>
- [58] NVIDIA. 2025. NVIDIA DGX Spark: A Grace Blackwell AI supercomputer on your desk. <https://www.nvidia.com/en-us/products/workstations/dgx-spark/>
- [59] OpenAI. 2024. Hello GPT-4o. <https://openai.com/index/hello-gpt-4o/>
- [60] OpenAI. 2024. Introducing OpenAI o1. <https://openai.com/o1/>
- [61] OpenAI. 2025. Introducing OpenAI o3 and o4-mini. <https://openai.com/index/introducing-o3-and-o4-mini/>.
- [62] Kun Qian, Yongqing Xi, Jiamin Cao, Jiaqi Gao, Yichi Xu, Yu Guan, Binzhang Fu, Xuemei Shi, Fangbo Zhu, Rui Miao, Chao Wang, Peng Wang, Pengcheng Zhang, Xianlong Zeng, Eddie Ruan, Zhiping Yao, Ennan Zhai, and Dennis Cai. 2024. Alibaba HPN: A Data Center Network for Large Language Model Training. In *Proceedings of the ACM SIGCOMM 2024 Conference* (Sydney, NSW, Australia) (*ACM SIGCOMM '24*). Association for Computing Machinery, New York, NY, USA, 691 – 706. <https://doi.org/10.1145/3651890.3672265>
- [63] Zhen Qin, Weigao Sun, Dong Li, Xuyang Shen, Weixuan Sun, and Yiran Zhong. 2024. Various lengths, constant speed: efficient language modeling with lightning attention. In *Proceedings of the 41st International Conference on Machine Learning* (Vienna, Austria) (*ICML '24*). JMLR.org, Article 1688, 19 pages.
- [64] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2024. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. *arXiv:2305.18290* [cs.LG] <https://arxiv.org/abs/2305.18290>

[65] Md Shafayat Rahman, Saptarshi Bhowmik, Yevgeniy Ryasnianskiy, Xin Yuan, and Michael Lang. 2019. Topology-custom UGAL routing on dragonfly. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (Denver, Colorado) (SC '19). Association for Computing Machinery, New York, NY, USA, Article 17, 15 pages. <https://doi.org/10.1145/3295500.3356208>

[66] Bitar Darvish Rouhani, Ritchie Zhao, Ankit More, Mathew Hall, Alireza Khodamoradi, Summer Deng, Dhruv Choudhary, Marius Cornea, Eric Dellinger, Kristof Denolf, Stosic Dusan, Vennugil Elango, Maximilian Golub, Alexander Heinecke, Phil James-Roxby, Dharmesh Jani, Gaurav Kolhe, Martin Langhammer, Ada Li, Levi Melnick, Maral Mesmakhos-roshahi, Andres Rodriguez, Michael Schulte, Rasoul Shafipour, Lei Shao, Michael Siu, Pradeep Dubey, Paulius Micikevicius, Maxim Naumov, Colin Verrilli, Ralph Wittig, Doug Burger, and Eric Chung. 2023. Microscaling Data Formats for Deep Learning. arXiv:2310.10537 [cs.LG] <https://arxiv.org/abs/2310.10537>

[67] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. arXiv:1707.06347 [cs.LG] <https://arxiv.org/abs/1707.06347>

[68] ByteDance Seed. 2025. Seed1.5-Thinking: Advancing Superb Reasoning Models with Reinforcement Learning. arXiv:2504.13914 [cs.CL] <https://arxiv.org/abs/2504.13914>

[69] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300 [cs.CL] <https://arxiv.org/abs/2402.03300>

[70] Noam Shazeer. 2019. Fast Transformer Decoding: One Write-Head is All You Need. CoRR abs/1911.02150 (2019). <http://arxiv.org/abs/1911.02150>

[71] Qwen Team. 2024. Qwen2.5 Technical Report. arXiv preprint arXiv:2412.15115 (2024).

[72] Qwen Team. 2025. Qwen3: Think Deeper, Act Faster. <https://github.com/QwenLM/Qwen3>

[73] Song Wang, Bing Yu, Wenwu Xiao, Fujun Bai, Xiaodong Long, Liang Bai, Xuerong Jia, Fengguo Zuo, Jie Tan, Yixin Guo, Peng Sun, Jun Zhou, Qiong Zhan, Sheng Hu, Yu Zhou, Yi Kang, Qiwei Ren, and Xiping Jiang. 2023. A 135 GBps/Gbit 0.66 pJ/bit Stacked Embedded DRAM with Multilayer Arrays by Fine Pitch Hybrid Bonding and Mini-TSV. In *2023 IEEE Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits)*. 1–2. <https://doi.org/10.23919/VLSITechnologyandCir57934.2023.10185427>

[74] xAI. 2024. Grok-2 Beta Release. <https://x.ai/news/grok-2>.

[75] xAI. 2024. Our Gigafactory of Compute:Colossus. <https://x.ai/colossus>.

[76] Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, Y. X. Wei, Lean Wang, Zhiping Xiao, Yuqing Wang, Chong Ruan, Ming Zhang, Wenfeng Liang, and Wangding Zeng. 2025. Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention. <https://arxiv.org/abs/2502.11089>

[77] Juntao Zhang, Haofeng Huang, Pengle Zhang, Jia Wei, Jun Zhu, and Jianfei Chen. 2025. SageAttention2: Efficient Attention with Thorough Outlier Smoothing and Per-thread INT4 Quantization. arXiv:2411.10958 [cs.LG] <https://arxiv.org/abs/2411.10958>

[78] Chenggang Zhao, Liang Zhao, Jiashi Li, and Zhean Xu. 2025. DeepGEMM: clean and efficient FP8 GEMM kernels with fine-grained scaling. <https://github.com/deepseek-ai/DeepGEMM>.

<https://github.com/deepseek-ai/DeepGEMM>.

[79] Chenggang Zhao, Shangyan Zhou, Liyue Zhang, Chengqi Deng, Zhean Xu, Yuxuan Liu, Kuai Yu, Jiashi Li, and Liang Zhao. 2025. DeepEP: an efficient expert-parallel communication library. <https://github.com/deepseek-ai/DeepEP>.

[80] Size Zheng, Jin Fang, Xuegui Zheng, Qi Hou, Wenlei Bao, Ningxin Zheng, Ziheng Jiang, Dongyang Wang, Jianxi Ye, Haibin Lin, Li-Wen Chang, and Xin Liu. 2025. TileLink: Generating Efficient Compute-Communication Overlapping Kernels using Tile-Centric Primitives. arXiv:2503.20313 [cs.DC] <https://arxiv.org/abs/2503.20313>

[81] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, Santa Clara, CA, 193–210. <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>

ISCA '25, 2025 年 6 月 21–25 日, 日本**赵威钢**, 邓承祺, 阮冲, 戴大斐, 高华祚, 李佳实, 张立越, 黄盼盼, 周上炎, 马世荣, 梁文锋, 何颖, 王雨晴, 刘宇轩, and Y.X. Wei

accepted 5 June 2009